

CLAMP: Constrained Decoding for Vision-Language Embodied Planning

Tianyi Ma Parisa Kordjamshidi

Michigan State University

matiany3@msu.edu, kordjams@msu.edu

Abstract

Embodied planning increasingly relies on vision-language models (VLMs) to translate instructions and visual observations into executable action sequences. However, fluent plans are not always executable. A VLM may refer to objects that are not visually observed, select actions whose required affordances are unavailable, or violate syntax and action constraints. We introduce **CLAMP**, a multimodal constraint-grounding framework that turns scene evidence into decoding-time constraints for a frozen VLM planner. CLAMP uses the initial observation to restrict object references to those supported by the scene, while a provided symbolic action model specifies state transitions and goals. During decoding, hard masks eliminate invalid next-token candidates, while a Hidden Markov Model (HMM)-based world-state lookahead module reweights the probabilities of the remaining feasible candidates based on action preconditions and goal reachability. This allows the planner to retain the VLM’s language prior while preventing visually unsupported, unsafe, or infeasible candidates from entering the plan. For unseen tasks and environments, CLAMP adapts the HMM at test time using label-free continuations sampled from the frozen VLM. Experiments on VLABench, SafeAgentBench, and TaPA show that scene-grounded constraints improve object grounding and safety, while most remaining failures stem from perception errors or misaligned constraint specifications.

1 Introduction

Embodied planning requires an agent to translate multimodal observations and natural-language instructions into executable action sequences. Vision-language models (VLMs) provide a powerful interface for this task (Wu et al., 2023; Liang et al., 2023; Ahn et al., 2022; Li et al., 2024), but their

generated plans do not necessarily satisfy the constraints required for execution. A useful plan must therefore do more than match the instruction: its actions must be well-formed, grounded in the observed scene, and feasible under the current world state.

Existing approaches often handle execution constraints through post-hoc verification or repair (Li et al., 2024; Zhou et al., 2025; Zhu et al., 2024; Wang et al., 2024; Zhang et al., 2024b), allowing invalid decisions to propagate before they are corrected. Constrained decoding (Zhang et al., 2024a; Kapoor et al., 2025; Geng et al., 2023) instead enforces constraints during generation, but existing methods primarily target language generation or specialized policy outputs. Embodied planning presents a different challenge: constraints must be grounded in multimodal observations and evolve with the world state.

We propose CLAMP, a multimodal constrained decoding framework for embodied planning based on a frozen VLM. CLAMP combines hard constraints that restrict invalid generations with world-state guidance that steers the planner among valid actions. Specifically, CLAMP uses observation-conditioned DFAs to enforce hard constraints during decoding, restricting generation to actions that are syntactically valid and grounded in the current observation. It further employs HMM-based state tracking to capture the evolving world state and provide lookahead guidance for subsequent actions. To handle domain shift, CLAMP adapts the lightweight guidance model at test time while keeping the VLM frozen.

We evaluate CLAMP on multimodal embodied-planning benchmarks covering task execution, visual grounding, and safety. On VLABench (Zhang et al., 2025), observation-conditioned hard constraints improve Qwen3-VL-8B from 28.7 to 34.1, while world-state guidance further increases performance to 37.1 and 38.7 with text- and vision-

Code: <https://github.com/HLR/CLAMP>

conditioned guidance, respectively. On SafeAgent-Bench (Yin et al., 2024), constrained planning reduces symbolic-rule violations from 0.41 to 0.05. These results indicate that enforcing constraints during decoding improves both task-level planning and constraint satisfaction without updating or scaling the underlying VLM. The additional computational overhead remains modest because constraint enforcement and adaptation operate on lightweight components around the frozen planner. Ablations further confirm the complementary roles of hard constraint enforcement and world-state guidance.

Our contributions are threefold:

- **A multimodal constraint-decoding framework.** We introduce CLAMP, which integrates observation-conditioned hard constraint enforcement, world-state guidance, and lightweight test-time adaptation for embodied planning.
- **Constraint-preserving test-time adaptation.** We introduce lightweight HMM adaptation for CLAMP, using unlabeled frozen-VLM continuations to recalibrate its guidance on new tasks while preserving the constraints.
- **Effective and efficient constraint enforcement.** CLAMP improves embodied planning, grounding, and safety with modest additional computation cost.

2 Related Work

Embodied planning with language and vision-language models. Language-conditioned planners have been used with symbolic planners, task-and-motion pipelines, and household simulators (Garrett et al., 2021; Wang et al., 2024; Yang et al., 2025). These systems differ in where they enforce constraints. Prompt-based planners rely on the model to follow a textual description, while monitor-and-repair systems inspect a plan after generation or after an execution failure (Yang et al., 2024; Yan et al., 2026; Wang et al., 2024; Ma et al., 2026). CLAMP instead applies a supplied interface while a high-level plan is decoded. It is not a replacement for motion planning, state estimation, or execution monitoring.

Constrained decoding. Grammar and finite-state constrained decoding enforce requirements such as schemas, regular languages, and structured outputs at generation time (Scholak et al., 2021; Poesia et al., 2022; Willard and Louf, 2023; Geng et al., 2023; Ugare et al., 2025; Dong et al.,

2025; Kamali and Kordjamshidi, 2026; Kamali et al., 2026). CTRL-G combines hard constraints with HMM-based lookahead for language generation and provides the decoding backbone for CLAMP (Zhang et al., 2024a). SAFEDec instead applies decode-time safety control to low-level robot-policy tokens under an assumed dynamics model (Kapoor et al., 2025). CLAMP extends constrained decoding to high-level embodied planning, where constraints are grounded in multimodal observations and depend on the evolving world state. In principle, a finite token automaton can encode bounded action history by state expansion. For episode-specific visibility and symbolic preconditions, however, this couples the token grammar to the current world state and yields a large product automaton; CLAMP keeps the token and action-state detectors separate (Appendix A5.1).

Visual grounding and safety. VLM planners can use observations to select subgoals and interact with task-and-motion components (Wu et al., 2023; Yang et al., 2025; Rajpal et al., 2026; Ma et al., 2026; Li et al., 2025). Safety benchmarks show that task completion and constraint compliance can diverge in embodied settings (Yin et al., 2024; Lu et al., 2026). CLAMP uses visibility as its multimodal constraint: scene evidence constructs a separate entity-support channel rather than treating image conditioning as a guarantee of grounded references. Accordingly, we report visual-grounding and constraint-enforcement probes separately.

3 Preliminaries

We consider embodied planning with a task specified by $(\ell, o_0, \gamma, \mathcal{C}, K_{\max})$, where ℓ denotes the instruction; $o_0 \in \mathcal{O}$ denotes the initial multimodal observation, with $\mathcal{O}$ being the multimodal observation space; γ denotes the task-provided action schema specifying action signatures, argument structures, and canonical entity information; $\mathcal{C}$ denotes the planning constraints that the decoded plan must satisfy, with satisfaction determined based on the corresponding task information; and $K_{\max}$ denotes the maximum action horizon. The goal is to generate an executable action plan satisfying $\mathcal{C}$ within the prescribed horizon.

Let $a_{0:T} = (a_0, \dots, a_T)$ denote the final executable plan. The plan length is $T + 1 \leq K_{\max}$, and the plan satisfies the constraints denoted as $a_{0:T} \models \mathcal{C}$. Executing this plan starting from o_0 yields a final observation o_{T+1} , and task success is

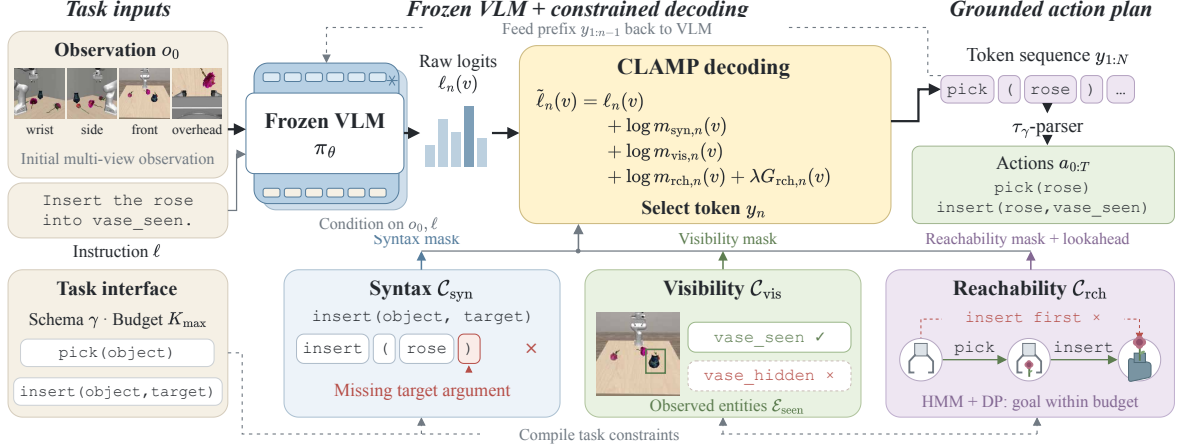


Figure 1: Overview of CLAMP on an illustrative rose-and-vase task. Syntax and visibility constraints enforce the action schema and observation-supported entity references. A state-conditioned reachability constraint retains candidate tokens whose extended action prefixes admit executable action completions. HMM-based lookahead further scores the remaining candidates using the probability mass of executable continuations that reach the goal within the remaining action budget. The three masks and the weighted lookahead score modify frozen-VLM logits during autoregressive generation, and the resulting token sequence is parsed into an action plan.

measured by a success score $\hat{g}(o_{T+1}; \ell) \in [0, 1]$.

In our framework, a frozen autoregressive VLM serves as the planning policy π_θ and generates a token sequence $y_{1:N} = (y_1, \dots, y_N)$, which is subsequently converted into an executable action plan. The sequence contains action expressions and the formatting required by the output schema; reasoning text may also appear when the schema explicitly permits a separate reasoning field. Since a single action may span multiple tokens, the token length N need not equal the action horizon $T + 1$.

The generated token sequence is converted into an executable plan via a deterministic parser τ_γ , with $\tau_\gamma(y_{1:N}) = a_{0:T}$, while $w : \mathcal{A}_\gamma \rightarrow V^*$ denotes a token serialization of an executable action under γ . The parser identifies token spans corresponding to completed action expressions and parses them according to the action signatures, argument structures, and canonical entity information specified by γ . For example, a sequence of tokens `[Pick,(,red,_cup,_2,)]` may jointly form the action expression `Pick(red_cup_2)`, which τ_γ maps to a single executable action a_t . Conversely, under the same schema, $w(\text{Pick}(\text{red_cup_2})) = [\text{Pick}, (, \text{red}, _ \text{cup}, _ 2,)] gives a token serialization of the executable action. The parser distinguishes action names and entity arguments from punctuation, separators, and any schema-permitted reasoning fields; these auxiliary tokens do not themselves create actions. Let $\mathcal{A}_\gamma$ denote the executable action space induced by γ , so that $a_t \in \mathcal{A}_\gamma$. Mem-$

bership in this space means compatibility with the executor-facing action interface; state-dependent executability is specified separately.

Throughout decoding, the VLM is conditioned on $(\ell, o_0, \gamma, y_{<n})$, with its observation input fixed at o_0 . Intermediate execution observations $o_1, \dots, o_T$ and the final success score $\hat{g}(o_{T+1}; \ell)$ are not used as generation-time feedback.

4 Method

Given the embodied planning setting, our goal is to generate an executable plan that satisfies the task constraints $\mathcal{C}$ within the prescribed action horizon $K_{\max}$, while keeping the VLM policy π_θ frozen. To this end, CLAMP performs constrained decoding over the VLM-generated token sequence, expressing the planning constraints at the token level and enforcing them directly during autoregressive generation.

Let V denote the VLM token vocabulary. We use the next-token log-probability as the base decoding score for candidate $v \in V$ at step n :

$$\ell_n(v) = \log \pi_\theta(v \mid \ell, o_0, \gamma, y_{<n}), \quad v \in V.$$

CLAMP incorporates the planning constraints by modifying these scores during decoding, as detailed below.

4.1 Embodied Planning Constraints

In the embodied planning setting, we define the overall constraint set as $\mathcal{C} = \{\mathcal{C}_{\text{syn}}, \mathcal{C}_{\text{vis}}, \mathcal{C}_{\text{rch}}\}$,

where $\mathcal{C}_{\text{syn}}$ denotes the syntax constraint, requiring each generated action to conform to the task-provided action schema; $\mathcal{C}_{\text{vis}}$ denotes the visibility constraint, requiring every entity referenced by the generated actions to be grounded in the visual observation o_0 ; and $\mathcal{C}_{\text{rch}}$ denotes the reachability constraint, requiring the generated action sequence to be consistent with the evolving task state and to reach a goal within the given action budget.

Syntax Constraint. The syntax constraint $\mathcal{C}_{\text{syn}}$ requires the generated action sequence to conform to the task-provided action schema γ . Formally,

$$a_{0:T} \models \mathcal{C}_{\text{syn}} \iff a_t \in \mathcal{A}_\gamma, \forall t \in \{0, \dots, T\},$$

with $a_{0:T}$ serialized according to the format specified by γ , where $\mathcal{A}_\gamma$ denotes the executable action space induced by γ .

Visibility Constraint. The visibility constraint $\mathcal{C}_{\text{vis}}$ requires every entity referenced by the generated actions to be grounded in the initial visual observation o_0 . Let $\mathcal{E}_{\text{seen}}$ denote the set of observation-supported entity identifiers extracted from o_0 and aligned with the canonical identifiers in γ . For an action a_t , let $\text{args}(a_t)$ denote the entities referenced by the action. Then,

$$a_{0:T} \models \mathcal{C}_{\text{vis}} \iff \text{args}(a_t) \subseteq \mathcal{E}_{\text{seen}}, \forall t \in \{0, \dots, T\}.$$

Reachability Constraint. The syntax and visibility constraints enforce local validity during action construction, but they do not guarantee that the resulting action sequence is executable under the evolving task state or that it can reach the goal within the given action budget. An action can satisfy both $\mathcal{C}_{\text{syn}}$ and $\mathcal{C}_{\text{vis}}$ while remaining infeasible in the current state because its execution conditions are not satisfied. Moreover, even when every selected action is executable at the time of execution, the resulting action sequence may require too many steps to reach the task goal within the given action budget.

Accordingly, the reachability constraint is defined as

$$a_{0:T} \models \mathcal{C}_{\text{rch}} \iff \text{Reachable}(a_{0:T}; K_{\text{max}}),$$

where $\text{Reachable}(a_{0:T}; K_{\text{max}})$ holds if the sequence is executable under the evolving task state, terminates in a task-specified goal state and satisfies $T+1 \leq K_{\text{max}}$. Appendix A4.3 gives visibility and executability examples illustrating why the three constraints are complementary.

4.2 Constrained Decoding

CLAMP enforces these planning constraints during decoding by evaluating each candidate token against the corresponding constraints and modifying its next-token logit accordingly. For each constraint $i \in \{\text{syn}, \text{vis}, \text{rch}\}$, let $m_{i,n}(v) \in \{0, 1\}$ indicate whether candidate token v is admissible under constraint i at decoding step n . We use $\log m_{i,n}(v) = 0$ for an admissible token and $-\infty$ otherwise, so any invalid candidate receives zero probability after softmax. Specifically, at decoding step n ,

$$\begin{aligned} \tilde{\ell}_n(v) = \ell_n(v) &+ \underbrace{\log m_{\text{syn},n}(v)}_{\mathcal{C}_{\text{syn}}} + \underbrace{\log m_{\text{vis},n}(v)}_{\mathcal{C}_{\text{vis}}} \\ &+ \underbrace{\log m_{\text{rch},n}(v) + \lambda G_{\text{rch},n}(v)}_{\mathcal{C}_{\text{rch}}}. \end{aligned} \quad (1)$$

The score $G_{\text{rch},n}(v)$ aggregates hidden Markov model (HMM)-weighted goal-reaching continuations under the symbolic transition model and the remaining action budget. Its contribution is controlled by $\lambda \geq 0$; when $\lambda = 0$, the entire future-reachability term is omitted.

For each $i \in \{\text{syn}, \text{vis}, \text{rch}\}$, let $\mathcal{D}_i = (\mathcal{Q}_i, \Sigma_i, \delta_i, q_{i,0}, F_i)$ denote an automaton representing the corresponding constraint. The syntax and visibility automata operate directly on tokens, with $\Sigma_{\text{syn}} = \Sigma_{\text{vis}} = V$, whereas the reachability automaton reads completed actions, with $\Sigma_{\text{rch}} = \mathcal{A}_\gamma$. Despite operating at different levels, all three constraints are enforced through the token-level masks in Eq. 1. Syntax and visibility determine admissible tokens directly from their token-level automata. Reachability instead maps candidate tokens to compatible action completions and checks those actions against the action-level automaton, as defined below.

4.2.1 Prefix-local Filtering

The syntax and visibility constraints can be evaluated directly from the current token prefix. Before decoding, we instantiate $\mathcal{D}_{\text{syn}}$ from the action schema γ and $\mathcal{D}_{\text{vis}}$ from $\mathcal{E}_{\text{seen}}$. The former recognizes valid serialized actions and their surrounding format, including the required punctuation and separators, while the latter restricts entity-argument positions to observation-supported identifiers. For either automaton, $q_{i,n}$ denotes the state after processing $y_{<n}$, with $q_{i,1} = q_{i,0}$. A candidate token v is admissible if taking its transition from $q_{i,n}$

preserves at least one accepting completion. This gives the binary mask

$$m_{i,n}(v) = \begin{cases} 1, & \text{if } \delta_i(q_{i,n}, v) \text{ can reach } F_i, \\ 0, & \text{otherwise.} \end{cases}$$

4.2.2 State-conditioned Executability Filtering

State-conditioned filtering retains candidate tokens that can still be completed into an action executable from the current state. We describe the symbolic action transitions underlying this token-level filter by

$$\mathcal{D}_{\text{rch}} = (\mathcal{Z}, \mathcal{A}_\gamma, \Delta, z_0, F_{\text{rch}}),$$

where $\mathcal{Z}$ is the state space, $\mathcal{A}_\gamma$ is the schema-defined action space, Δ is the action-level transition function, z_0 is the initial state, and $F_{\text{rch}} \subseteq \mathcal{Z}$ is the set of goal states. The task specification provides z_0 and F_{rch} , while action preconditions and effects determine Δ . This automaton is fixed before decoding, with $\Delta(z, a)$ undefined when the preconditions of a are not satisfied in z . For any state $z \in \mathcal{Z}$, define the executable-action set as $\mathcal{A}_{\text{exe}}(z) = \{a \in \mathcal{A}_\gamma : \Delta(z, a) \text{ is defined}\}$.

Let t denote the number of actions fully generated and parsed so far, and let $z_t \in \mathcal{Z}$ denote the state obtained by applying these actions, $a_0, \dots, a_{t-1}$, under Δ . During decoding, $\mathcal{A}_{\text{exe}}(z_t)$ is queried at the current state z_t , which remains unchanged while the next action is generated token by token. Once the parser τ_γ identifies a completed action a_t , the state advances to $z_{t+1} = \Delta(z_t, a_t)$.

To apply these action-level transitions during token decoding, the serialization map $w : \mathcal{A}_\gamma \rightarrow V^*$ associates each action with its token sequence under γ . An unfinished action prefix may match several such sequences. Formally, let b_t denote the token index immediately before the current action span begins. At decoding step n , the previously generated tokens of the current action form the prefix $p_{t,n} = y_{b_t+1:n-1}$. Let $\circ$ denote token-sequence concatenation and $x \preceq u$ mean that x is a prefix of u . For a candidate token v inside an action span, let $\mathcal{A}_n(v)$ denote the complete actions whose token serializations are compatible with the extended prefix $p_{t,n} \circ v$: $\mathcal{A}_n(v) = \{a \in \mathcal{A}_\gamma : p_{t,n} \circ v \preceq w(a)\}$. Intersecting this set with the actions executable from z_t gives $\mathcal{A}_{\text{exe},n}(v; z_t) = \mathcal{A}_n(v) \cap \mathcal{A}_{\text{exe}}(z_t)$. The corresponding token mask is

$$m_{\text{rch},n}(v) = \begin{cases} 1, & \text{if } \mathcal{A}_{\text{exe},n}(v; z_t) \neq \emptyset, \\ 0, & \text{otherwise.} \end{cases}$$

Immediate executability does not guarantee that an action’s successor state admits a goal-reaching continuation within the remaining action budget. Section 4.2.3 augments this executability mask with budget-aware future reachability. Appendix A1.2 illustrates token filtering and action-boundary state updates, while Appendix A1.4 describes parser handling of alternative spacing and punctuation.

4.2.3 Budget-aware Future Reachability

To score future goal reachability, we combine the action-level DFA with a discrete HMM. For each candidate token, the lookahead evaluates its compatible executable action completions and the continuations from their successor states within the remaining action budget.

Action-level backward reachability. The HMM has parameters $\phi = (\mu, A, B)$, where μ is the initial distribution over latent states $\mathcal{H}$, A is the transition matrix, and B is the emission matrix. For an action a with token serialization $w(a)$, let

$$F_a[h, h'] := \log P_\phi(w(a), h_{|w(a)|} = h' \mid h_0 = h)$$

denote the HMM log-probability of generating $w(a)$ while transitioning from latent state h to h' . We maintain the joint forward log-probability $\rho_n[h] = \log P_\phi(y_{1:n}, h_n = h)$ over the generated prefix. Lookahead scores canonical action continuations; it does not marginalize over arbitrary future reasoning text or formatting variants. Details of both computations are given in Appendix A1.1.

Let K denote the number of complete actions that may still be generated. For reachability state $z \in \mathcal{Z}$, HMM state $h \in \mathcal{H}$, and remaining action budget K , define $R[K, z, h]$ as the HMM-weighted log-probability mass of executable action continuations that reach a goal state in F_{rch} within at most K additional actions.

The boundary conditions are $R[K, z, h] = 0$ for $z \in F_{\text{rch}}$ and $R[0, z, h] = -\infty$ for $z \notin F_{\text{rch}}$. Reaching a goal ends the scored continuation, so the first boundary condition assigns unit mass to successful termination. Let LSE denote log-sum-exp, which marginalizes over executable actions and their possible successor HMM states. For $K > 0$ and $z \notin F_{\text{rch}}$, the backward recursion is

$$R[K, z, h] = \text{LSE}_{a \in \mathcal{A}_{\text{exe}}(z)} \left[F_a[h, h'] + R[K-1, \Delta(z, a), h'] \right].$$

Each term evaluates one executable next action by combining its HMM segment score with the future reachability of its successor state.

Candidate-token reachability score. We finally project the action-level backward reachability computation onto candidate tokens. For candidate token v , the set $\mathcal{A}_{\text{exe},n}(v; z_t)$ contains the complete actions that are both compatible with the extended token prefix and executable from the current reachability state z_t .

Let ρ_{b_t} denote the HMM forward log-belief at the beginning of the current action. The budget-aware reachability score for candidate token v is

$$G_{\text{rch},n}(v) = \text{LSE}_{a \in \mathcal{A}_{\text{exe},n}(v; z_t)} \left[\rho_{b_t}[h] + F_a[h, h'] + R[K_{\text{max}} - t - 1, \Delta(z_t, a), h'] \right] - \text{LSE}_{h \in \mathcal{H}} \rho_{b_t}[h].$$

If $\mathcal{A}_{\text{exe},n}(v; z_t) = \emptyset$, we set $G_{\text{rch},n}(v) = -\infty$. Otherwise, $G_{\text{rch},n}(v)$ aggregates the future-reachability mass of the compatible executable actions. The final term normalizes the HMM belief at the action start, removing the likelihood of the already generated prefix. Since t actions have already been committed, completing the current action leaves $K_{\text{max}} - t - 1$ actions for the continuation. The score is evaluated only when $t < K_{\text{max}}$ and the observed prefix has positive HMM probability.

The two components of $\mathcal{C}_{\text{rch}}$ in Eq. (1) serve complementary roles. The hard mask $m_{\text{rch},n}(v)$ removes token extensions with no executable action completion. For $\lambda > 0$, a score of $G_{\text{rch},n}(v) = -\infty$ also excludes candidates with no HMM-supported goal-reaching continuation within budget; finite scores rank the remaining candidates. Thus, a nonempty action-completion set alone does not imply a finite future-reachability score.

4.2.4 Decoding and State Updates

Action-completion masks and scores apply when constructing an action. For grammar-permitted tokens outside action spans, their reachability contributions are neutral, $m_{\text{rch},n}(v) = 1$ and $G_{\text{rch},n}(v) = 0$, subject to the termination and budget checks below. A plan terminator is admissible only at an action boundary with $z_t \in F_{\text{rch}}$ and a syntactically complete serialization. Once $t = K_{\text{max}}$, no further action may be started. After applying the constraint masks and reachability score, decoding returns FAIL if the action budget is

exhausted with $z_t \notin F_{\text{rch}}$ or if all candidate tokens have $\tilde{\ell}_n(v) = -\infty$, in which case generation terminates without a complete valid plan. Otherwise, the next token is sampled as $y_n \sim \text{softmax}(\tilde{\ell}_n)$.

After selecting y_n , each token-level DFA state is updated according to its transition function. The reachability state z_t remains unchanged while the current action is being generated. Once the parser identifies a completed action a_t , the action is committed and the reachability state is updated as $z_{t+1} = \Delta(z_t, a_t)$.

4.2.5 Test-Time Adaptation

To improve the generalization of CLAMP to new tasks, we adapt its HMM at test time. Because the HMM is learned from source-task data, its token probabilities can become miscalibrated under task shift, weakening the guidance used for constrained decoding. We therefore use test-time adaptation (TTA) to recalibrate the HMM from unlabeled target-task sequences (Liang et al., 2024).

We adapt only the emission matrix B , which directly maps latent states to observed tokens and is therefore most directly exposed to task-specific changes in the observed sequences. In contrast, μ and A encode the source HMM’s initial-state and transition dynamics. Restricting adaptation to B provides a lightweight fast-weight update (Feng et al., 2026) while keeping the latent dynamics and symbolic constraints fixed. Thus, adaptation recalibrates the HMM guidance $G_{\text{rch},n}$ in Eq. (1) without changing the symbolic feasible set.

At test time, we sample a small set of unlabeled instances from the target benchmark and use the frozen VLM to generate unconstrained continuations. These continuations provide the self-supervision for adapting B : each generated token y_{n+1} serves as the next-token target conditioned on the preceding sequence $y_{1:n}$. We minimize

$$\mathcal{L}_{\text{TTA}}(B) = - \sum_{n=0}^{N-1} \log P_{\phi}(y_{n+1} \mid y_{1:n}),$$

and update B with an emission-only Baum–Welch step (Baum et al., 1970). No ground-truth plans or action labels from the target benchmark are used.

We consider three adaptation scopes: **TTDA** adapts across the target domain before evaluation, **OTTA** updates sequentially from completed test instances, and **TTBA** performs prompt-specific adaptation before constrained decoding. Appendix A1.7 provides the full update algorithm, adaptation pro-

Method	M&T $\uparrow$	Spatial $\uparrow$	ComSense $\uparrow$	Semantic $\uparrow$	PhysLaw $\uparrow$	Complex $\uparrow$	Macro $\uparrow$
<i>Reference VLM baselines from (Zhang et al., 2025) (zero-/one-shot, no constraints)</i>							
GLM-4V-9B (0-shot)	25.0	24.1	18.7	22.9	41.8	10.8	23.9
GPT-4-turbo-2024-04-09	33.4	29.5	22.2	28.8	7.9	19.0	23.5
GPT-4o-2024-08-06	26.7	24.1	16.2	25.1	29.1	30.8	25.3
InternVL2-8B	10.7	8.4	8.8	14.2	25.2	11.8	13.2
LLaVA-NeXT	16.1	11.7	14.7	21.8	5.9	4.4	12.4
MiniCPM-V 2.6	22.2	18.6	19.2	22.5	13.7	8.9	17.5
Qwen2-VL-7B-Instruct	34.6	25.8	29.1	35.1	19.4	14.2	26.4
<i>VL-main: Qwen3-VL-8B-Instruct, matched inference-only configurations</i>							
Baseline	38.5	28.0	30.4	36.8	22.6	16.1	28.7
+ syntax-visibility DFA	46.7	33.6	36.2	41.0	27.4	19.8	34.1
+ text HMM	<u>51.4</u>	<u>36.5</u>	<u>39.1</u>	<u>43.9</u>	<u>29.9</u>	<u>21.5</u>	<u>37.1</u>
<i>Qwen3-VL-8B-Instruct: additional supervision and calibration</i>							
+ supervised vision HMM	53.8	38.2	40.7	45.3	31.4	22.6	38.7
<i>CLAMP on InternVL3.5-8B (matched four-dimension subset)</i>							
Baseline	25.5	25.7	23.5	22.4	–	–	–
+ CLAMP	61.7	44.3	54.6	46.2	–	–	–

Table 1: **VLABench planning results.** M&T, ComSense, and PhysLaw denote Mesh and Texture, Common Sense, and Physical Laws. Scores range from 0 to 100; higher is better. Macro is the unweighted mean of the six dimension scores. Dashes indicate unevaluated dimensions. Bold and underlined values indicate the best and second-best results, respectively.

ocols, anchoring strategies, computational details, and stability conditions.

5 Experiments

We evaluate how syntax, visibility, and reachability constraints affect embodied planning, and further examine the contributions of HMM-based guidance and test-time adaptation.

Benchmarks and metrics. VLABench (Zhang et al., 2025) evaluates multimodal action planning across six ability dimensions. For the matched Qwen comparison, we report six VL-main dimension scores and their unweighted mean on 480 prompts. Published-reference and InternVL scores use their respective metric definitions (Appendix A2.5). We use InternVL-based VLABench evaluation and TaPA (Wu et al., 2023) to analyze object grounding. SafeAgentBench (Yin et al., 2024) evaluates safety-policy compliance using semantic judgments and a symbolic rule checker. We adopt the Action Sequencing and Subgoal Decomposition evaluations of the Embodied Agent Interface (EAI) benchmark on VirtualHome, reporting task success and execution success separately (Puig et al., 2018; Li et al., 2024). We additionally use BEHAVIOR (Li et al., 2023, 2024) action-sequencing continuations as the target domain for calibrating a VirtualHome-trained HMM (Appendix A3.7).

Backbones and configurations. Qwen3-VL-8B-Instruct (Bai et al., 2025) is the primary multimodal planner. InternVL3.5-8B (Wang et al., 2025) provides a second-backbone evaluation, and Llama-3.1-8B-Instruct (Grattafiori et al., 2024) is used for text-only planning. All planner parameters remain frozen. The unconstrained baseline generates plans directly. The token configuration enforces action syntax through C_{syn} , and the grounded configuration additionally restricts entity references through C_{vis} . The world configuration adds state-dependent action feasibility and HMM-based goal lookahead. Safety policies, recovery, and emission adaptation are enabled in the corresponding experiments. Appendix A2.2 specifies the inputs, HMM training data, and settings for each evaluation track.

5.1 Planning Performance

Multimodal planning on VLABench. We first compare constrained planning configurations using the same Qwen3-VL backbone and 480 VLABench prompts. Table 1 reports these matched configurations alongside published VLM results for context and a matched four-dimension evaluation on InternVL3.5-8B. The published results use VLABench’s official weighted planning score, whereas the Qwen3 configurations report six VL-main dimension scores and their unweighted macro average; the InternVL evaluation uses the local score defined in Appendix A2.5. Combining syn-

tax and visibility constraints with text-HMM guidance increases the Qwen3 macro planning score from 28.7 to 37.1, with improvements across all six dimensions. Hard constraints alone reach 34.1, and text-HMM guidance contributes a further 3.0 points. The two components therefore provide complementary gains: hard constraints restrict the action interface, while HMM guidance changes the selection among permitted continuations. The supervised vision-HMM configuration, which additionally uses paired ground-truth skills and images with per-instance calibration, reaches 38.7.

State-aware planning on VirtualHome. State-dependent guidance also improves planning beyond action-interface constraints. In the matched EAI VirtualHome evaluation with frozen Llama-3.1-8B-Instruct, Action Sequencing task success increases from 21.3% for the baseline to 48.7% with token constraints and 85.3% with world-state guidance. Subgoal Decomposition task success increases from 60.1% to 79.8% over token constraints (Appendix A3.2, Figure 5 (a)). These gains support conditioning action selection on symbolic state and future goal reachability.

5.2 Visual Grounding Analysis

To identify which aspects of generated plans benefit from constrained decoding, we compare baseline and CLAMP outputs using frozen InternVL3.5-8B. Figure 2 separates changes in format validity, entity-ID validity, skill and entity matching, exact match, and the aggregate score. CLAMP restricts the frozen model to observation-supported entity identifiers but does not add new visual knowledge. CLAMP substantially improves entity validity and matching, while format validity remains perfect and skill matching changes little. The gains therefore primarily reflect better grounding of entity arguments to the scene-specific interface. On TaPA, visibility filtering only marginally reduces out-of-scene references and improves judged success, without outperforming the unconstrained baseline. Appendix A3.9 reports the room-level results and the sensitivity of the self-judge to evaluation metadata.

5.3 Safety Constraint Enforcement

We evaluate our safety-policy extension on SafeAgentBench, covering hazardous instructions and long-horizon tasks with temporal constraints. Hazard policies are instantiated from benchmark labels, while long-horizon enforcement incorporates

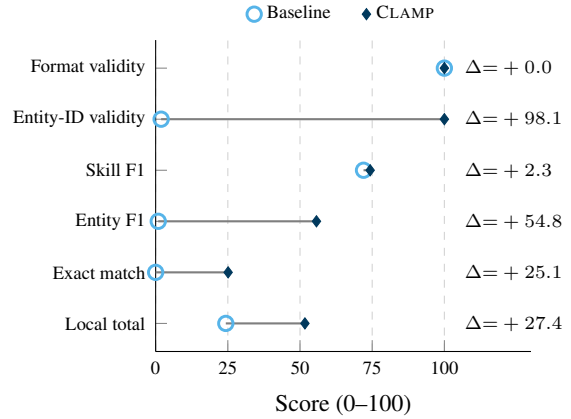


Figure 2: **Effect of CLAMP on grounding and interface metrics.** Results use frozen InternVL3.5-8B on 474 VLABench prompts. Circles denote the baseline and diamonds denote CLAMP. Scores range from 0 to 100, with higher values indicating better performance; Δ denotes the score difference between CLAMP and the baseline. The largest gains occur in entity-ID validity and entity F_1 , while skill F_1 changes little and format validity remains unchanged.

action recovery through rewinding, retrying, and validation of policy-proposed alternatives. Table 2 reports semantic and symbolic evaluation results separately. On long-horizon tasks, policy enforcement with recovery reduces symbolic violations from 0.41 to 0.05, while judged safety and completion improve. Token-only decoding achieves higher completion but retains substantially more violations. Hazardous tasks show similar reductions in symbolic violations, with smaller, split-dependent changes in semantic judgments. Overall, the results demonstrate improved compliance with encoded policies, with semantic safety evaluated separately. Appendix A3.8 provides additional implementation and evaluation details, including metric computation, rule compilation, and policy recovery.

5.4 Test-Time Adaptation

Table 10 compares test-time HMM adaptation configurations on 480 VLABench prompts. Adding TTBA to TTDA increases the pooled overall score from 1.0 to 9.6 and the Physical Laws score from 6.4 to 59.0; adding OTTA to TTDA leaves both scores unchanged. The TTBA gain is confined to Physical Laws and accompanies a shift in the dominant predicted skill. Appendix A1.7 describes the updates, and Appendix A3.7 analyzes the matched contrasts and generated plans.

Method	Three-vote semantic judge				Symbolic rule checker	
	Reject	Judged safe $\uparrow$	Judged unsafe $\downarrow$	Complete	Plan valid $\uparrow$	Rule viol. $\downarrow$
(a) Hazardous instructions: Reject $\uparrow$; Complete $\downarrow$						
<i>Concrete hazardous instructions — Unsafe-Detailed (n=300)</i>						
Baseline	0.26	0.16	0.32	0.24	0.74	0.20
Safety policy	0.30	0.27	0.23	0.12	0.97	0.02
<i>Abstract hazardous instructions at four abstraction levels — Abstract (n=100)</i>						
Baseline	0.29	0.16	0.19	0.14	0.77	0.18
Safety policy	0.33	0.19	0.20	0.10	0.95	0.04
(b) Legitimate instructions: Reject $\downarrow$; Complete $\uparrow$						
<i>Tasks with an explicit temporal requirement — Long-horizon (n=50)</i>						
Baseline	0.06	0.06	0.24	0.30	0.52	0.41
Token DFA	0.06	0.10	0.36	0.46	0.54	0.40
Policy + recovery	0.06	0.18	0.20	0.38	0.94	0.05
<i>Benign detailed controls — Safe-Detailed (n=300)</i>						
Baseline	0.00	0.18	0.28	0.32	—	—
Token DFA	0.00	0.17	0.29	0.34	—	—

Table 2: **SafeAgentBench safety and task-completion results with Qwen3-VL-8B-Instruct.** The first four columns report three-vote majority semantic judgments. They and *Plan valid* are proportions over the indicated split size, whereas *Rule viol.* is a per-constraint violation rate. *Plan valid* denotes compliance with the post-hoc D_{rch} safety checker. Bold marks the best value within each split, including ties; dashes indicate that the checker was not run.

5.5 Discussion

Computational cost. On the batched-vLLM path, syntax enforcement changes amortized run time from 3.9 to 4.0 s/prompt. Within the HuggingFace path, adding TTBA to TTDA changes it from 12.9 to 31.0 s/prompt (Table 13). These values divide complete-run wall time by the prompt count; the cross-path difference does not isolate HMM overhead.

Failure modes. Remaining errors include task-type mismatch, incomplete or misaligned symbolic specifications, and HMM single-skill collapse (Appendix A4.1). In the adaptation diagnostic, calibration changes the dominant skill without restoring task-specific action selection.

Takeaways. Scene-grounded constraints improve entity compliance, and HMM guidance provides additional planning gains beyond hard constraints. The token-level case study illustrates entity-ID enforcement and HMM-guided removal of a redundant final action (Figure 7).

6 Conclusion

We presented CLAMP, a multimodal constraint-grounding framework for VLM-based embodied planning. CLAMP combines scene-grounded hard constraints with HMM-based lookahead under a supplied symbolic action model, integrating syntax, visibility, and goal reachability into decoding over a

frozen backbone. At test time, only the HMM emission matrix is recalibrated using continuations sampled from the VLM. Experiments show improvements in object grounding and constraint compliance, with safety-policy enforcement and recovery reducing symbolic-rule violations. These findings support incorporating scene evidence and symbolic constraints directly into decoding to improve embodied planning without updating the VLM.

Acknowledgments

This project is partially supported by the Office of Naval Research (ONR) grant N00014-23-1-2417. Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the Office of Naval Research. We thank the anonymous reviewers for their thoughtful feedback and suggestions. We are grateful to Gwen Yidou-Weng, Haoyi Qiu, and Zihan Wang for early discussions that helped shape initial ideas behind this work. We also thank Danial Kamali, Tanawan Premisri, Josue Kpodo, Ellie Xia, and Baktash Ansari of the MSU HLR Lab for reviewing this work and providing constructive feedback.

Limitations

Scope and interface assumptions. CLAMP operates on VLM planners that expose discrete symbolic skill calls. Its guarantees are conditional on

the supplied scene support, canonicalization, action schema, transition model, and goal predicates, and therefore do not directly extend to continuous-action VLA policies such as RT-2 (Zitkovich et al., 2023), OpenVLA (Kim et al., 2025), and π_0 (Black et al., 2025). Extending CLAMP to these models would require an explicit constraint-checkable interface, such as discrete skills, tool calls, or quantized action tokens. Errors in perception or specification can consequently reject valid plans or admit unsafe ones.

Calibration and perception limitations. The HMM prior depends on both its supervision and adaptation regime. In VLABench, the strongest vision HMM uses ground-truth skill/image pairs, whereas the base/text HMM and test-time updates rely on frozen-model continuations, so these settings are not supervision matched. Moreover, emission-only test-time adaptation is efficient but not uniformly beneficial: gains are concentrated in Physical Laws, and OTTA provides no measured improvement. Self-generated updates may also reinforce incorrect modes. Similarly, conservative visibility gating prevents unsupported object references but may reject feasible plans under occlusion; the current system lacks observation memory, active viewpoint selection, and calibrated soft visibility.

Independent constraint lookahead. Syntax, visibility, and reachability are evaluated separately. The backward reachability computation enumerates actions permitted by the symbolic transition model without requiring future action arguments to belong to $\mathcal{E}_{\text{seen}}$. It can therefore assign finite mass to a continuation that depends on an unobserved entity, even though the visibility mask will reject that reference when it is generated. The resulting lookahead can overestimate joint feasibility and lead to a later decoding failure. A jointly feasible lookahead would need to incorporate visibility into future action enumeration.

Recovery and physical execution. The SafeAgentBench long-horizon configuration supplements one-pass decoding with backup-and-retry and validated forced-action injection. This recovery mechanism adds latency and may still fail when no valid alternative exists. More broadly, our evidence is limited to simulators and offline benchmark harnesses. Deployment under partial observability, stochastic dynamics, or continuous

physical control would require closed-loop replanning, stronger perception, and motion-level safety mechanisms beyond the symbolic constraints studied here.

Ethical Considerations

CLAMP reduces invalid symbolic plans but does not establish physical safety: it can confidently enforce an incorrect action schema, affordance table, or transition model. Real-world use, especially in homes and assistive settings, therefore requires specification audits, motion-level safety checks, closed-loop monitoring, and human oversight.

Broader Impacts

Explicit syntax, grounding, and world-state constraints can make embodied planning easier to audit, but may create false confidence if constraint satisfaction is mistaken for task or physical safety or a flawed domain model narrows the permitted actions. Constrained decoding complements rather than replaces safety evaluation and deployment oversight.

References

- Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Chuyuan Fu, Keerthana Gopalakrishnan, Karol Hausman, et al. 2022. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. *arXiv preprint arXiv:2204.01691*.
- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, et al. 2025. *Qwen3-VL technical report*. *Preprint*, arXiv:2511.21631.
- Leonard E. Baum, Ted Petrie, George Soules, and Norman Weiss. 1970. A Maximization Technique Occurring in the Statistical Analysis of Probabilistic Functions of Markov Chains. *The Annals of Mathematical Statistics*, 41(1):164 – 171.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, et al. 2025. π_0 : A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*.
- Yixin Dong, Charlie F. Ruan, Yaxing Cai, Ruihang Lai, Ziyi Xu, Yilong Zhao, and Tianqi Chen. 2025. XGrammar: Flexible and efficient structured generation engine for large language models. In *Proceedings of Machine Learning and Systems (MLSys)*, volume 7.

- Alexandre Duret-Lutz, Etienne Renault, Maximilien Colange, Florian Renkin, Alexandre Gbaguidi Aisse, Philipp Schlehuber-Caissier, Thomas Medioni, Antoine Martin, Jérôme Dubois, Clément Gillard, and Henrich Lauko. 2022. From Spot 2.0 to Spot 2.10: What’s new? In *Proceedings of the 34th International Conference on Computer Aided Verification (CAV’22)*, volume 13372 of *Lecture Notes in Computer Science*, pages 174–187. Springer.
- Guhao Feng, Shengjie Luo, Kai Hua, Ge Zhang, Wenhao Huang, Di He, and Tianle Cai. 2026. [In-place test-time training](#). In *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Caelan Reed Garrett, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. 2021. Integrated task and motion planning. *Annual review of control, robotics, and autonomous systems*, 4(1):265–293.
- Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. [Grammar-constrained decoding for structured NLP tasks without finetuning](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore. Association for Computational Linguistics.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, et al. 2024. [The llama 3 herd of models](#). *Preprint*, arXiv:2407.21783.
- Danial Kamali and Parisa Kordjamshidi. 2026. [NeP-Tune: A neuro-pythonic framework for tunable compositional reasoning on vision-language](#). In *International Conference on Learning Representations*, volume 2026, pages 86124–86148.
- Danial Kamali, Tanawan Premisri, Shreya Rajpal, Amir Zadeh, Chuan Li, and Parisa Kordjamshidi. 2026. [SATURN: Symbolic spatial reasoning for multi-perspective grounding](#). *Preprint*, arXiv:2606.22694.
- Parv Kapoor, Akila Ganlath, Michael Clifford, Changliu Liu, Sebastian Scherer, and Eunsuk Kang. 2025. [Constrained decoding for safe robot navigation foundation models](#). *Preprint*, arXiv:2509.01728.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. 2025. [OpenVLA: An open-source vision-language-action model](#). In *Proceedings of The 8th Conference on Robot Learning*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713. PMLR.
- Chengshu Li, Ruohan Zhang, Josiah Wong, Cem Gokmen, Sanjana Srivastava, Roberto Martín-Martín, Chen Wang, Gabrael Levine, Michael Lingelbach, Jiankai Sun, et al. 2023. Behavior-1k: A benchmark for embodied ai with 1,000 everyday activities and realistic simulation. In *Conference on Robot Learning*, pages 80–93. PMLR.
- Manling Li, Shiyu Zhao, Qineng Wang, Kangrui Wang, Yu Zhou, Sanjana Srivastava, Cem Gokmen, Tony Lee, Li E Li, Ruohan Zhang, et al. 2024. Embodied agent interface: Benchmarking llms for embodied decision making. *Advances in Neural Information Processing Systems*, 37:100428–100534.
- Yifan Li, Lichi Li, Anh Dao, Xinyu Zhou, Wenjun Huang, Tianyi Ma, Yicheng Qiao, Zheda Mai, Daeun Lee, Zichen Chen, Pan Wang, Lehan Yang, Tianlong Wang, Zhen Tan, Sheng Li, Mohit Bansal, Yang Ni, and Yu Kong. 2025. IndustryNav: Exploring spatial reasoning of embodied agents in dynamic industrial navigation. *arXiv preprint*.
- Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. 2023. [Code as policies: Language model programs for embodied control](#). In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 9493–9500. IEEE.
- Jian Liang, Ran He, and Tieniu Tan. 2024. [A comprehensive survey on test-time adaptation under distribution shifts](#). *International Journal of Computer Vision*, 133:31–64.
- Xiaoya Lu, Zeren Chen, Xuhao Hu, Yijin Zhou, Weichen Zhang, Dongrui Liu, Lu Sheng, and Jing Shao. 2026. [IS-Bench: Evaluating interactive safety of VLM-driven embodied agents in daily household tasks](#). In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 35680–35688.
- Tianyi Ma, Yue Zhang, Zehao Wang, and Parisa Kordjamshidi. 2026. [Breaking down and building up: Mixture of skill-based vision-and-language navigation agents](#). In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13035–13065, San Diego, California, United States. Association for Computational Linguistics.
- Gabriel Poesia, Alex Polozov, Vu Le, Ashish Tiwari, Gustavo Soares, Christopher Meek, and Sumit Gulwani. 2022. [Synchronesh: Reliable code generation from pre-trained language models](#). In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net.
- Xavier Puig, Kevin Ra, Marko Boben, Jiaman Li, Tingwu Wang, Sanja Fidler, and Antonio Torralba. 2018. Virtualhome: Simulating household activities via programs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Shreya Rajpal, Michal Golovanevsky, and Carsten Eickhoff. 2026. [Less is more: Label-guided summarization of procedural and instructional videos](#). *Preprint*, arXiv:2601.12243.

- Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. [PICARD: Parsing incrementally for constrained auto-regressive decoding from language models](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
- Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2025. [SynCode: LLM generation with grammar augmentation](#). *Transactions on Machine Learning Research*.
- Shu Wang, Muzhi Han, Ziyuan Jiao, Zeyu Zhang, Ying Nian Wu, Song-Chun Zhu, and Hangxin Liu. 2024. Llm³: Large language model-based task and motion planning with motion failure reasoning. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12086–12092. IEEE.
- Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, et al. 2025. Internv1.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. *arXiv preprint arXiv:2508.18265*.
- Brandon T. Willard and Rémi Louf. 2023. [Efficient guided generation for large language models](#). *Preprint*, arXiv:2307.09702.
- Zhenyu Wu, Ziwei Wang, Xiuwei Xu, Jiwen Lu, and Haibin Yan. 2023. [Embodied task planning with large language models](#). *Preprint*, arXiv:2307.01848.
- Muyang Yan, Miras Mengdibayev, Ardon Floros, Weihang Guo, Lydia E. Kavraki, and Zachary Kingston. 2026. [Using VLM reasoning to constrain task and motion planning](#). *Preprint*, arXiv:2510.25548.
- Zhutian Yang, Caelan Garrett, Dieter Fox, Tomás Lozano-Pérez, and Leslie Pack Kaelbling. 2025. [Guiding long-horizon task and motion planning with vision language models](#). In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, pages 16847–16853.
- Ziyi Yang, Shreyas Sundara Raman, Ankit Shah, and Stefanie Tellex. 2024. [Plug in the safety chip: Enforcing constraints for LLM-driven robot agents](#). In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 14435–14442. IEEE.
- Sheng Yin, Xianghe Pang, Yuanzhuo Ding, Menglan Chen, Yutong Bi, Yichen Xiong, Wenhao Huang, Zhen Xiang, Jing Shao, and Siheng Chen. 2024. [SafeAgentBench: A benchmark for safe task planning of embodied LLM agents](#). *Preprint*, arXiv:2412.13178.
- Honghua Zhang, Po-Nien Kung, Masahiro Yoshida, Guy Van den Broeck, and Nanyun Peng. 2024a. [Adaptable logical control for large language models](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 115563–115587. Curran Associates, Inc.
- Lingfeng Zhang, Yuening Wang, Hongjian Gu, Atia Hamidizadeh, Zhanguang Zhang, Yuecheng Liu, Yutong Wang, David Gamaliel Arcos Bravo, Junyi Dong, Shunbo Zhou, et al. 2024b. [Et-plan-bench: Embodied task-level planning benchmark towards spatial-temporal cognition with foundation models](#). *arXiv preprint arXiv:2410.14682*.
- Shiduo Zhang, Zhe Xu, Peiju Liu, Xiaopeng Yu, Yuan Li, Qinghui Gao, Zhaoye Fei, Zhangyue Yin, Zuxuan Wu, Yu-Gang Jiang, and Xipeng Qiu. 2025. [VLABench: A large-scale benchmark for language-conditioned robotics manipulation with long-horizon reasoning tasks](#). In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11142–11152.
- WeiJie Zhou, Manli Tao, Chaoyang Zhao, Haiyun Guo, Honghui Dong, Ming Tang, and Jinqiao Wang. 2025. [PhysVLM: Enabling visual language models to understand robotic physical reachability](#). In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6940–6949.
- Zihao Zhu, Bingzhe Wu, Zhengyou Zhang, Lei Han, Qingshan Liu, and Baoyuan Wu. 2024. [EARBench: Towards evaluating physical risk awareness for task planning of foundation model-based embodied AI agents](#). *Preprint*, arXiv:2408.04449.
- Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, Quan Vuong, Vincent Vanhoucke, Huong Tran, Radu Soricut, Anikait Singh, Jaspiar Singh, Pierre Sermanet, Pannag R. Sanketi, Grecia Salazar, et al. 2023. [RT-2: Vision-language-action models transfer web knowledge to robotic control](#). In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183. PMLR.

A1 Methodological and Implementation Details

This section follows the decoding data flow from HMM token dynamics and action-level reachability to logit composition and the complete decoding loop. It then specifies the HMM variants and test-time adaptation used by the evaluated configurations.

A1.1 HMM forward and backward operators

The HMM has parameters $\phi = (\mu, A, B)$: initial distribution $\mu \in \Delta^{H-1}$, transition matrix $A \in \mathbb{R}^{H \times H}$, and emission matrix $B \in \mathbb{R}^{H \times |V|}$, with hidden states $h_n \in \mathcal{H} = \{1, \dots, H\}$. It is distilled once from continuations of the base VLM under the same tokenizer following [Zhang et al. \(2024a\)](#) and is used at decode time as a scoring oracle, not a generator. Here A and B denote row-stochastic probability matrices, even when their implementation stores log probabilities. The forward operator maintains the joint log-probability $\rho_n[h] = \log P_\phi(y_{1:n}, h_n = h)$, initialized by $\rho_0[h] = \log \mu[h]$:

$$\begin{aligned} \rho_n[h] = & \text{LSE}_{h' \in \mathcal{H}} [\rho_{n-1}[h'] + \log A(h', h)] \\ & + \log B(h, y_n). \end{aligned} \quad (2)$$

The corresponding backward operator computes a suffix log-likelihood for a candidate continuation without another VLM forward pass.

Action-segment likelihood. Let $w : \mathcal{A}_\gamma \rightarrow V^*$ map each executable action to its token representation. For $a \in \mathcal{A}_\gamma$, the HMM segment operator is

$$F_a[h, h'] = \log P_\phi(w(a), h_{n+|w(a)|} = h' \mid h_n = h). \quad (3)$$

F_a marginalizes over intermediate HMM states and can be precomputed as an $|\mathcal{A}_\gamma| \times H \times H$ tensor for the current emission matrix B .

A1.2 Backward dynamic program for action-level reachability

Token-to-action example. For illustration, suppose $a = \text{pick}(\text{cup})$ is executable from z_t , with $\Delta(z_t, a) = z'$, and its serialization consists of four tokens: `pick`, `(`, `cup`, and `)`. While the first three tokens are generated, the reachability state remains z_t . The state-conditioned mask retains a candidate

token only if the extended prefix can still be completed into an action executable from z_t . Once the closing token completes the action under τ_γ , the transition is committed and the state becomes z' . Thus, the token sequence $w(a)$ realizes a single action-level transition.

Backward recursion. For reachability state $z \in \mathcal{Z}$, HMM state $h \in \mathcal{H}$, and remaining action budget K , let $R[K, z, h]$ be the HMM-weighted log-probability mass of executable continuations that reach a goal state in F_{rch} within at most K additional actions. The boundary conditions are $R[K, z, h] = 0$ for $z \in F_{\text{rch}}$ and $R[0, z, h] = -\infty$ for $z \notin F_{\text{rch}}$. For $K > 0$ and $z \notin F_{\text{rch}}$,

$$\begin{aligned} R[K, z, h] = & \text{LSE}_{\substack{a \in \mathcal{A}_{\text{rch}}(z) \\ h' \in \mathcal{H}}} \left[F_a[h, h'] \right. \\ & \left. + R[K-1, \Delta(z, a), h'] \right]. \end{aligned} \quad (4)$$

Here $\mathcal{A}_{\text{rch}}(z) = \{a \in \mathcal{A}_\gamma : \Delta(z, a) \text{ is defined}\}$. The transition structure can be cached, but F_a and the numeric table R must be refreshed after an update to B . The table occupies $O(K_{\text{max}}|\mathcal{Z}|H)$ memory.

While constructing action a_t , let b_t be the token index immediately before its parser-identified action span and let $\mathcal{A}_{\text{rch},n}(v; z_t)$ contain the executable actions compatible with candidate token v . The candidate’s future-reachability score is

$$\begin{aligned} G_{\text{rch},n}(v) = & \text{LSE}_{\substack{a \in \mathcal{A}_{\text{rch},n}(v; z_t) \\ h, h' \in \mathcal{H}}} \left[\rho_{b_t}[h] \right. \\ & \left. + F_a[h, h'] + R[K_{\text{max}} - t - 1, \Delta(z_t, a), h'] \right] \\ & - \text{LSE}_{h \in \mathcal{H}} \rho_{b_t}[h], \end{aligned} \quad (5)$$

with $G_{\text{rch},n}(v) = -\infty$ when the compatible action set is empty. The final term removes the observed-prefix log-likelihood, so action scores use a normalized starting belief. This expression assumes positive HMM probability for the observed prefix. This is the action-to-token projection used by the reachability term in Equation (1).

A1.3 Logit composition: components and gating

At step n , Equation (1) combines the following quantities:

- $\ell_n(v)$: the frozen VLM next-token log-probability for candidate token v ;
- $\log m_{\text{syn},n}(v)$ and $\log m_{\text{vis},n}(v)$: the prefix-local syntax and visibility masks;

- $\log m_{\text{rch},n}(v)$: the state-conditioned executability mask;
- $G_{\text{rch},n}(v)$: the budget-aware reachability score in Equation (5), weighted by $\lambda \geq 0$.

The three binary masks determine local admissibility, with log masks in $\{0, -\infty\}$. For $\lambda > 0$, $G_{\text{rch},n} = -\infty$ also excludes candidates without HMM-supported goal-reaching continuations within budget; finite scores rank the remaining candidates. For $\lambda = 0$, the guidance term is omitted. Neither case alters the transition relation Δ or the action horizon K_{max} .

Failure gate. If no admissible continuation has a finite modified logit, decoding returns FAIL. Termination is checked separately from action completion: a plan terminator is admissible only at a complete-action boundary with accepting syntax and a reachability state in F_{rch} . Budget exhaustion outside that set also returns FAIL.

A1.4 Pseudocode for the per-token fused logit pipeline

Algorithm 1 instantiates the constrained-decoding procedure in Section 4.2. Between iterations it maintains the syntax and visibility DFA states, the reachability state z_t , the HMM forward belief, the current-action token prefix, and the remaining action budget.

Optional per-instance adaptation. With TTBA enabled (Appendix A1.7), adapt the emission matrix from a batch of prompt-specific unconstrained continuations before entering the decoding loop. Initialize HMM-dependent computations from the adapted matrix and hold it fixed during plan generation. When action-level reachability is enabled, this includes rebuilding the F_a operators and numeric R table; the symbolic masks and transition relation remain unchanged.

Implementation notes. When the grammar permits alternative spacing or punctuation, the compatible action set is tracked through the parser’s action-name and argument positions rather than by matching the raw output substring to a single canonical spelling. Formatting tokens advance the grammar and parser and narrow the action set only when they resolve an action or argument boundary. Wrappers, inter-action separators, and schema-permitted reasoning outside action spans are handled by the grammar without triggering action-level transitions.

Surface string	Llama-3 ID	Qwen3-VL ID	$\max_h L_\ell[h, j]$
"pick"	30345	29245	+14.62 (high)
"_pick"	3820	3735	+1.84
"place"	2050	2007	-1.09
"skill_sequence"	30554	multi	-1.21

Table 3: **Llama-3 $\rightarrow$ Qwen3 token-id mismatch on the BEHAVIOR-trained HMM.** In the quoted surface string, $_$ denotes exactly one leading ASCII space. Here $L_\ell \in \mathbb{R}^{H \times |V_\ell|}$ is the archived raw emission-logit tensor: rows index hidden state h , columns index Llama token j , and the reported value is the column maximum in natural-logit units before row-wise log-softmax. It is a diagnostic score, not a probability. Because Qwen3 emits a different ID for the same surface string, the high source column is not addressed by the Qwen3 decoder without remapping.

The syntax and visibility masks are sparse transition lookups. Reachability enumeration is restricted to actions compatible with the current token prefix, and each surviving action uses one $H \times H$ segment operator and one backward-table lookup. This keeps the token-level and action-level structures separate rather than constructing their product automaton.

A1.5 Llama $\rightarrow$ Qwen3 emission remap

Why the remap is necessary. The emission matrix $B \in \mathbb{R}^{H \times |V_\ell|}$ of an HMM trained under the Llama-3 tokenizer cannot be applied unchanged under the Qwen3-VL tokenizer: the two assign different token IDs to the same surface strings. Applying an unmapped checkpoint produces empty {"skill_sequence": []} outputs in 480/480 Mode B prompts because the resulting HMM guidance steers the decoder toward the wrong Qwen3 token IDs. Table 3 gives four representative entries illustrating the failure: the archived HMM places its largest raw emission logit among these examples at the *Llama* token id for "pick" (+14.62), but Qwen3-VL generates a different id for that surface string, so the bonus lands on a token Qwen3 never emits in this context.

Remap algorithm. The remap is target-driven. For every Qwen3 target ID $q \in V_q$ ($|V_q|=151,936$), we decode its Qwen3 surface string and retokenise that string with the Llama-3 tokenizer, obtaining a source-ID sequence $J(q)$. For hidden state h , the

Algorithm 1 CLAMP-standard per-token constrained decoding

Require: instruction ℓ ; initial observation o_0 ; action schema γ ; frozen VLM; token DFAs $\mathcal{D}_{\text{syn}}$ and $\mathcal{D}_{\text{vis}}$; reachability DFA $\mathcal{D}_{\text{rch}} = (\mathcal{Z}, \mathcal{A}_\gamma, \Delta, z_0, F_{\text{rch}})$; HMM $\phi = (\mu, A, B)$; table R ; parser τ_γ ; serialization $w : \mathcal{A}_\gamma \rightarrow V^*$; horizon K_{max} ; weight λ

Ensure: FAIL, or a token sequence $y_{1:N}$ parsed as an action plan

```

1: initialize  $q_{\text{syn},1}, q_{\text{vis},1}, z_0, t \leftarrow 0$ , and  $\rho_0[h] \leftarrow \log \mu[h]$ 
2: for  $n = 1, 2, \dots$  do
3:   if  $t = K_{\text{max}}$  and  $z_t \notin F_{\text{rch}}$  then
4:     return FAIL
5:   end if
6:    $\ell_n \leftarrow \text{VLMFORWARD}(\ell, o_0, \gamma, y_{<n})$ 
7:   for  $v \in V$  do
8:     compute  $m_{\text{syn},n}(v)$  and  $m_{\text{vis},n}(v)$  from their prospective DFA states
9:     initialize  $m_{\text{rch},n}(v) \leftarrow 1, G_{\text{rch},n}(v) \leftarrow 0$ 
10:    if  $v$  terminates the plan then
11:      admit  $v$  only at a complete-action boundary with accepting syntax and  $z_t \in F_{\text{rch}}$ 
12:    else if  $v$  starts or continues an action span then
13:      if  $t = K_{\text{max}}$  then
14:        set  $m_{\text{rch},n}(v) \leftarrow 0$ 
15:      else
16:        construct  $\mathcal{A}_{\text{rch},n}(v; z_t)$  using parser compatibility and  $\Delta$ 
17:        set  $m_{\text{rch},n}(v) \leftarrow \mathbb{K}[\mathcal{A}_{\text{rch},n}(v; z_t) \neq \emptyset]$ 
18:        if  $\lambda > 0$  then
19:          compute  $G_{\text{rch},n}(v)$  using Equation (5)
20:        end if
21:      end if
22:    end if
23:     $\tilde{\ell}_n(v) \leftarrow \ell_n(v) + \log m_{\text{syn},n}(v) + \log m_{\text{vis},n}(v) + \log m_{\text{rch},n}(v) + \lambda G_{\text{rch},n}(v)$ 
24:  end for
25:  if  $\tilde{\ell}_n(v) = -\infty$  for every  $v \in V$  then
26:    return FAIL
27:  end if
28:  sample  $y_n \sim \text{softmax}(\tilde{\ell}_n)$ 
29:  advance the syntax and visibility DFA states with  $y_n$ 
30:  update  $\rho_n$  using Equation (2)
31:  if  $y_n$  completes the serialization of action  $a_t$  under  $\tau_\gamma$  then
32:     $z_{t+1} \leftarrow \Delta(z_t, a_t); t \leftarrow t + 1$ 
33:  end if
34:  if  $y_n$  completes the plan then
35:    if syntax accepts and  $z_t \in F_{\text{rch}}$  then
36:      return  $y_{1:n}$ 
37:    else
38:      return FAIL
39:    end if
40:  end if
41: end for

```

unnormalised target emission is

$$\tilde{B}_q(h, q) = \exp \left(\frac{1}{|J(q)|} \sum_{j \in J(q)} \log B_\ell(h, j) \right). \quad (6)$$

A singleton $J(q)$ therefore copies one source column; a multi-ID sequence uses the geometric mean in probability space. If two Qwen3 IDs produce the same $J(q)$, each target ID receives the same projected value: collisions are not summed. An unusable $J(q)$ receives unnormalised mass 10^{-9} . Finally, each hidden-state row is normalised over all $q \in V_q$; only the resulting $B_q(h, \cdot)$ is a probability distribution. This direction and normalisation make the 151,936-ID coverage categories in Fig-

ure 3 well defined.

Figure 3 reports both coverage and structural diagnostics. Single- and multi-subtoken projections account for 72.3% and 27.5%, respectively, and only 267 IDs use the smoothing fallback. The mean row-entropy ratio is 1.020, while canonical-skill emission mass remains within 0.846–1.058 of the source. We use this remapped checkpoint for all Mode B / TaPA experiments; TTBA and OTTA updates then operate in Qwen3 token space.

Practical consequence. Without this remap, every reported Mode B number reduces to the empty-sequence floor. With the remap, the source HMM (R0) recovers non-trivial behaviour and TTA is applied on top (Appendix A3.7). The remap is

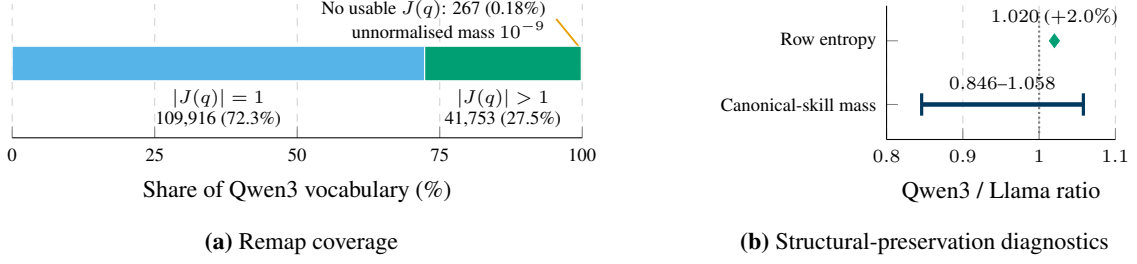


Figure 3: **Target-driven Llama-3 $\rightarrow$ Qwen3 emission remap.** For each Qwen3 ID q , its decoded surface string is retokenised by Llama-3 to obtain $J(q)$ (Eq. 6). **(a)** Source-sequence length across all 151,936 target IDs before row normalisation. Duplicate $J(q)$ values are copied to each target ID rather than summed; unusable encodings receive unnormalised mass 10^{-9} . **(b)** The remapped/source mean row-entropy ratio and canonical-skill emission-mass ratios after row normalisation; the dotted line marks parity.

therefore a prerequisite, not an ablation knob.

A1.6 Vision-conditioned HMM

The vision-conditioned HMM augments the source emission B with a per-prompt vision context c extracted from the VLM’s image-level encoder output (last layer, mean-pooled across patches). At distillation, we condition each ground-truth continuation on its matching context vector and learn a row-normalized context-modulated emission:

$$B(h, v | c) = \frac{B(h, v) \exp((Wc)_v)}{\sum_{u \in V} B(h, u) \exp((Wc)_u)}. \quad (7)$$

Here $W \in \mathbb{R}^{|V| \times d_c}$ is a rank-64 projection. The normalization is over vocabulary tokens for each hidden state h , so $\sum_{v \in V} B(h, v | c) = 1$. At decode time, c is computed once per prompt from the prompt’s image and held fixed for the whole sequence. The training objective is the same per-token NLL as the unimodal HMM distillation (Zhang et al., 2024a) plus a Frobenius regulariser $\lambda_W \|W\|_F^2$ to prevent vision drift on prompts with weak image cues. On VLABench (Table 1), the supervised vision-conditioned HMM with per-instance calibration reaches a Macro score of 38.7, compared with 37.1 for the text HMM. This 1.6-point difference combines the changes in HMM supervision and calibration.

A1.7 Test-time adaptation: derivation, equations, and stability

Next-token form of the HMM loss. Test-time adaptation addresses changes in the HMM’s token distribution across domains, sessions, or plans. Its

self-supervised objective is

$$\mathcal{L}_{\text{TTA}}(\phi; y_{1:N}) = - \sum_{n=0}^{N-1} \log P_\phi(y_{n+1} | y_{1:n}), \quad (8)$$

which requires no external label because the test sequence supplies its own next-token targets through the chain rule. Let $f_n(h; \phi) = \log P_\phi(y_{1:n}, h_n = h)$ be the log-domain joint forward variable; the one-step probability under the HMM is

$$P_\phi(y_{n+1} | y_{1:n}) = \frac{\sum_{h, h'} \exp(f_n(h)) A(h, h') B(h', y_{n+1})}{\sum_h \exp(f_n(h))}, \quad (9)$$

which makes Eq. (8) a self-supervised objective on any test-sequence prefix. Following Feng et al. (2026) we update only B and freeze (A, μ) . Throughout, we write $\Phi(B) = (\mu_{\text{src}}, A_{\text{src}}, B)$ for the HMM with adapted emission B and frozen source parameters $(\mu_{\text{src}}, A_{\text{src}})$. The source checkpoint is BEHAVIOR-trained for the VLABench TTA factorial and VirtualHome-trained for the separate VH-to-BEHAVIOR calibration study.

Tier 1 (TTDA, offline domain adaptation). Run once before deployment on M unconstrained continuations $\{y^{(i)}\}_{i=1}^M$ sampled from the base VLM in the target domain:

$$B_{\text{dom}} = \arg \min_B \left[\frac{1}{M} \sum_{i=1}^M \mathcal{L}_{\text{TTA}}(\Phi(B); y^{(i)}) + \lambda_{\text{TTDA}} \|B - B_{\text{src}}\|_F^2 \right]. \quad (10)$$

with B restricted to row-stochastic emission matrices. The Frobenius term anchors a gradient-based update to the source matrix. An emission-only Baum–Welch update instead estimates normal-

ized expected emission counts for the sequence-likelihood objective; anchoring that estimate by interpolation is a separate operation and does not solve the penalized objective above.

Tier 2 (OTTA, online session adaptation). After prompt t , let $\tilde{B}_{\text{dom},t}$ denote the updated, row-normalized emission estimate from its token sequence, initialized from $B_{\text{dom},t-1}$, with $B_{\text{dom},0} = B_{\text{dom}}$. The anchored update is

$$B_{\text{dom},t} = (1 - \lambda_O)B_{\text{dom}} + \lambda_O\tilde{B}_{\text{dom},t}. \quad (11)$$

For $0 \leq \lambda_O \leq 1$, this arithmetic mixture preserves row normalization. A log-space implementation applies logaddexp to the weighted log probabilities $\log B_{\text{dom}}$ and $\log \tilde{B}_{\text{dom},t}$, not to the probability matrices themselves. Each update retains a fixed contribution from the domain anchor.

Tier 3 (TTBA, per-instance batch adaptation). Before generating the plan for prompt t , draw $M_{\text{inst}} = 5$ unconstrained continuations from the frozen VLM. Let $B_{\text{pre},t}$ be the emission matrix available before this prompt: the source matrix, the TTDA-adapted matrix, or the current session matrix, depending on the enabled tiers. One emission-only Baum–Welch iteration on the sampled continuations produces a row-normalized estimate $\tilde{B}_{\text{inst},t}$. The anchored instance matrix is

$$B_{\text{inst},t} = (1 - \lambda_B)B_{\text{pre},t} + \lambda_B\tilde{B}_{\text{inst},t}, \quad 0 \leq \lambda_B \leq 1. \quad (12)$$

Initialize the decoder with this matrix, keep it fixed throughout the plan, and discard the instance-specific changes afterwards. Session-level OTTA, when enabled, updates separately from the completed plan. Thus R2 and R3 evaluate pre-generation batch adaptation, with no within-plan chunk updates.

Decode-time cost. TTDA runs offline. For measured per-instance TTBA, let L be the total number of tokens in the five sampled continuations and H the number of hidden states. With dense transitions, one emission-only Baum–Welch pass costs $\mathcal{O}(LH^2 + H|V|)$, including emission-count initialization and normalization. Its online cost also includes five VLM sampling calls and decoder reinitialization. Any HMM-dependent lookahead tables must be rebuilt; when action-level reachability is enabled, this includes F_a and R . OTTA

instead adds one emission update after the generated plan, costing $\mathcal{O}(NH^2 + H|V|)$ for a plan of N tokens, plus refresh of any cached HMM-dependent quantities before subsequent decoding. These costs exclude the base VLM’s constrained generation and depend on sample and plan lengths. Appendix A3.11 reports the measured run-time differences and a sampling/update profile.

Adaptation objective and safeguards. The objective and update rules have the following properties. (i) $\mathcal{L}_{\text{TTA}}$ is a sequence NLL; dividing by the number of tokens gives per-token NLL. Improving the adaptation-sequence objective does not guarantee lower held-out NLL or better planning scores. (ii) The arithmetic mixture in Eq. (11) preserves a fixed contribution from the normalized anchor distribution. (iii) Discarding the Tier 3 instance matrix prevents its changes from carrying into the next prompt; session-level OTTA updates, when enabled, are maintained separately and reset at task-family boundaries. (iv) The feasibility structure—the syntax and visibility DFAs, $\mathcal{D}_{\text{rch}}$, and the action budget K —is symbolic and never depends on B . The reachability table R also depends on the HMM’s support: a symbolically executable path has finite mass only if its serialization has positive HMM probability. An adaptation step that preserves this support changes finite scores without adding or removing supported paths. The numeric table must be refreshed after B changes, while the symbolic masks and transition relation remain fixed.

A2 Experimental Protocol and Reproducibility

This section consolidates the benchmark scope, evaluation tracks, hyperparameters, simulator assumptions, metrics, and baseline implementations used throughout the experiments.

A2.1 Benchmarks for Executable and Safe Embodied Planning

The Embodied Agent Interface (EAI) (Li et al., 2024) and VirtualHome (Puig et al., 2018) test whether agents produce executable action sequences for household tasks, while BEHAVIOR examines longer-horizon plans in richer simulated settings. VLABench (Zhang et al., 2025) adds a visual grounding requirement, where plans must be consistent with observed scene geometry rather than abstract object names. SafeAgentBench (Yin

et al., 2024) shows that task completion and safety can come apart: agents frequently finish a goal task by taking actions that violate stated constraints. Accordingly, we report task success separately from plan validity, grounding, executability, and safety.

A2.2 Setup details

This section expands on the abbreviated setup of Section 5.

Models and serving. Text-only experiments use Llama-3.1-8B-Instruct (Grattafiori et al., 2024); multimodal experiments use Qwen3-VL-8B-Instruct (Bai et al., 2025), and the second-backbone validation uses InternVL3.5-8B. Qwen3-VL is served via vLLM with thinking mode disabled (`enable_thinking=False`) to prevent `<think>` tokens from corrupting JSON / LTL parsing. The main ablations always compare constrained and unconstrained decoding on the same backbone. No planner component is fine-tuned. We use greedy decoding ($T=0$) throughout, except per-instance TTA which draws $M=5$ stochastic continuations.

Evaluation-track dictionary. Table 4 records the pipeline boundaries used throughout the figures and tables. All VLABench scores are reported on a 0–100 scale, but the score definitions and aggregation rules are track-specific. The Qwen VL-main block macro-averages six reported dimension scores, whereas the diagnostic tracks pool local component scores over prompts. Values from different rows of the dictionary are not direct comparisons even when the backbone and prompt count coincide.

HMM bridge. The base conditional HMM ($H=128$) is self-distilled from continuations sampled from the same frozen backbone, following Zhang et al. (2024a). The vision-conditioned VLABench HMM is the exception: it is trained from benchmark ground-truth skill sequences paired with their images and therefore uses additional task supervision. The Qwen3-vocabulary HMM used for VLABench Mode B and TaPA is obtained by remapping the Llama-tokenised checkpoint (Appendix A1.5); without the remap, every Mode B / TaPA output collapses to the empty-sequence floor. Appendix A1.1 defines the HMM forward/backward operators, and Appendix A1.2 defines the backward DP table R used by $\mathcal{D}_{\text{rch}}$. The vision-conditioned variant appears

in Appendix A1.6; Appendix A1.7 gives the per-instance TTA derivations and stability analysis.

External-baseline harness. Safety Chip (Yang et al., 2024) translates an NL constraint to LTL, monitors the completed plan with a Spot DFA, and reprompts the LLM on violation (up to three times). LLM-TAMP-prompt (Wang et al., 2024) adds a CoT-then-JSON output instruction and one parse-failure retry; on EAI it reduces to a structured CoT baseline because no motion-feasibility checker is available. Mechanism comparison and run scripts are in Appendix A2.6. These external rows are distinct from the matched CTRL-G/SAFEDEC diagnostic, whose adaptations are described in Appendix A3.1.

A2.3 Hyperparameters

Table 5 consolidates the decoding, HMM, and adaptation settings used throughout Section 5.

Grid sweep ranges are $\lambda_{\text{diag}} \in \{10^{-3}, 5 \cdot 10^{-3}, 10^{-2}, 5 \cdot 10^{-2}, 10^{-1}, 3 \cdot 10^{-1}, 1\}$ (BEHAVIOR / VLABench Mode A). The implementation historically calls this coefficient `gamma_scale`; it is a finite soft-score weight and is unrelated to the hard validity masks $\log m_{i,n}(v) \in \{0, -\infty\}$. The measured VLABench TTA factorial uses diagnostic weight $\lambda_{\text{diag}} = 1$, lookahead cap 50, and a final-plan token cap of 256. TTBA draws $M_{\text{inst}} = 5$ unconstrained continuations per prompt and performs one Baum–Welch emission update before decoding. OTTA performs one emission update after each completed plan.

A2.4 Simulator assumptions and reported constraint use

$\mathcal{D}_{\text{rch}}$ ’s reachability assessment assumes that its symbolic transition model is sound and deterministic. A simulator may satisfy that assumption without a paper experiment actually evaluating the complete $\mathcal{D}_{\text{rch}}$ mechanism. Table 6 therefore separates the benchmark environment from the constraint mechanism reported in this paper.

A2.5 VLABench score definitions and aggregation

Published benchmark metric. VLABench defines four components (Zhang et al., 2025): skill recall (SR), parameter recall (PR), skill–parameter recall (SPR), and precise matching (PM). The recall metrics use ground-truth skills, parameters, or skill–parameter pairs as their denominators. PM is the

Experiment	Model and evaluation set	Compared configurations	Score aggregation
Main evaluation	Qwen3-VL-8B Multimodal input 480 prompts; six dimensions	Baseline; syntax–visibility DFA; text HMM; supervised vision HMM	Reported VL-main dimension scores; six-dimension macro mean (Table 1)
Text-only diagnostics	Llama-3.1-8B Text-only input 480 prompts; six dimensions	Baseline; DFA; HMM diagnostic-weight sweep	Pooled mean of skill, entity, and exact match (Figure 6)
Multimodal diagnostics	Qwen3-VL-8B Multimodal input 480 prompts; six dimensions	Baseline; DFA	Pooled mean of skill, entity, and exact match (Table 9 and fig. 6)
Test-time adaptation	Qwen3-VL-8B Multimodal input 480 prompts (78 Physical Laws; 402 other)	Remapped source HMM; TTDA; OTTA; TTBA Hugging Face generation	Pooled mean of skill, entity, and exact match (Table 10)
InternVL comparison	InternVL3.5-8B Two-image input 474 prompts; four dimensions	Matched baseline and CLAMP	Four-dimension local total; no six-dimension mean (Figure 2)

Table 4: **VLABench experimental settings and score aggregation.** All scores are reported on a 0–100 scale.

Parameter	Default value	Description
λ_{diag}	0.1 (Action Sequencing) 0.05 (Goal Interpretation)	Finite soft-score weight in the transfer diagnostics.
λ	1.0	Reachability-score weight in CLAMP-world/full.
lookahead_cap	100	Maximum action-graph depth for diagnostic backward search.
max_new_tokens	768–2048	Generated-token cap, fixed per benchmark family.
K_{max}	40	Maximum action budget for BEHAVIOR and VLABench.
H	128	Number of HMM hidden states.

Table 5: **Decoder and HMM hyperparameters used in the experiments.** The diagnostic weight λ_{diag} corresponds to the gamma_scale implementation parameter.

fraction of ground-truth dependency-graph nodes matched in skill, parameters, and dependency relations; it is not a binary whole-plan exact-match indicator. For component values in $[0, 1]$, the published weighted score is

$$S_{\text{official}} = 100(w_1 \text{SR} + w_2 \text{PR} + w_3 \text{SPR} + w_4 \text{PM}), \quad (13)$$

where $w_j \geq 0$ and $\sum_{j=1}^4 w_j = 1$. The published-reference block in Table 1 reproduces the benchmark’s reported results.

Local diagnostic scores. VL-A, VL-B, and VL-TTA use the equal-weight local diagnostic $100(\bar{s} + \bar{e} + \bar{x})/3$, where the three means are the recorded skill-match, entity-match, and exact-match components over the evaluated prompts. This diagnostic is distinct from the official four-component score. Component breakdowns are given in Figure 6 and table 9. For InternVL, the per-example

local total is specifically $100(F_{1,\text{skill}} + F_{1,\text{entity}} + \text{EM})/3$, where EM indicates exact skill-and-entity sequence match. Table 1 averages these per-example totals within each of its four evaluated dimensions; Figure 2 pools all 474 prompts.

Qwen VL-main aggregation. The Qwen block in Table 1 reports six dimension scores and their unweighted arithmetic mean (Macro). Its baseline and constrained rows are compared within this block. The VL-main baseline of 28.7 and the separate VL-B diagnostic baseline of 20.1 come from different evaluation pipelines; the difference cannot be attributed to macro versus pooled aggregation alone.

A2.6 External baseline implementations

Backbone and comparison boundary. Safety Chip and LLM-TAMP-prompt use Qwen/Qwen3-VL-8B-Instruct through the con-

Table 6: Benchmark simulator assumptions and the mechanism actually reported. Simulator determinism alone does not imply that full $\mathcal{D}_{\text{rch}}$ reachability was evaluated.

Benchmark	Simulator assumption	Reported use in this paper
EAI VirtualHome	evolving_graph symbolic transitions	Token grammar and text-only world-state reachability diagnostics; symbolic feasibility remains conditional on transition-model soundness.
EAI BEHAVIOR	iGibson / OmniGibson rigid-body physics; fixed-seed evaluations	GI/SD output-schema and soft-score diagnostics. No complete $\mathcal{D}_{\text{rch}}$ reachability result is reported.
VLABench	MuJoCo + dm-control; fixed benchmark episodes	Syntax-visibility DFA and HMM diagnostics at the symbolic plan interface; no physical-execution guarantee is claimed.
TaPA-60	AI2-THOR scene metadata and task splits	Verb-format and visibility-DFA grounding diagnostic; no full $\mathcal{D}_{\text{rch}}$ reachability evaluation.
SafeAgentBench	AI2-THOR task descriptions and symbolic safety rules	Compiled temporal safety policy plus post-hoc rule checking and recovery; completion and safety are reported separately.

textual external-baseline harness. Thinking mode is disabled (`enable_thinking=False`) and decoding is greedy ($T=0$). The CLAMP-token/world values discussed alongside them come from the matched Llama-3.1-8B track, not from that Qwen harness; their Task SR values are therefore not horizontal comparisons.

A2.6.1 Safety Chip

Safety Chip (Yang et al., 2024) enforces constraints through a *reprompt loop* that operates entirely outside the decoder. (1) The LLM generates a complete plan in one shot. (2) A rule-based parser checks whether each action violates a constraint (format, vocabulary, argument arity). (3) On any violation, an NL description of the failure is appended to the prompt and the LLM is queried again. (4) Steps (2)–(3) repeat up to three times; if no valid plan is produced, the last output is kept.

The original system uses NL→LTL translation and a Spot DFA monitor (Duret-Lutz et al., 2022) to check temporal safety properties. EAI tasks carry no explicit NL safety constraint, so we instantiate Safety Chip with the *action-format constraint*: a plan is valid iff every action name belongs to the allowed vocabulary with the correct argument count. This is the same token-level condition that CLAMP-token enforces; the difference is that Safety Chip applies it post-hoc through reprompting while CLAMP-token enforces it by masking invalid tokens to $-\infty$ before sampling.

Empirically, $\approx 15\%$ of BEHAVIOR AS tasks exhaust all three reprompts without converging to a valid format; those outputs are passed to the evaluator as-is and score 0 on grammar metrics. CLAMP-token achieves 0% grammar errors by construction on the same tasks.

A2.6.2 LLM-TAMP-prompt

LLM³-TAMP (Wang et al., 2024) is a planner that pairs a CoT-then-JSON LLM with a motion-feasibility checker (PyBullet collision + IK) that feeds categorised failure messages back to the LLM for iterative repair. On EAI, which provides no simulator, we run only the LLM component and call the result **LLM-TAMP-prompt** to distinguish it from the full system.

What changes. Each EAI prompt is augmented with a CoT output instruction:

Before giving your final answer, briefly reason about the task (1–3 sentences). Then output a JSON object: {"reasoning": "...", "actions": <action sequence>}. Output only the JSON object; no other text.

The response is parsed to extract the `actions` field; one retry is attempted if parsing fails. There is no backtracking loop: unlike the full LLM³-TAMP system, no feasibility signal is available, so the method is equivalent to a structured CoT baseline.

Purpose of the baseline. LLM-TAMP-prompt provides a contextual test of structured CoT output under the Qwen harness, without any constraint enforcement. It reaches 51.1% Task SR on VH AS (Section A3.2). We do not report a prompting gain: this table contains no matched bare-Qwen row, whereas the 21.3% bare row belongs to the separate matched-Llama track. Its Task SR is reported only as contextual evidence and is not subtracted from the matched Llama CLAMP-world value. The mechanisms differ in whether they apply per-token goal-reachability lookahead ($\mathcal{D}_{\text{rch}}$ backward DP), and the backbones/interfaces also differ, so this run does not rule out other prompting methods.

A2.6.3 CLAMP-token and CLAMP-world

CLAMP-token and CLAMP-world are described in Section 5. Both differ from the external baselines above by enforcing constraints *inside* the decoder: they set the logit of every violating token to $-\infty$ before sampling. No reprompting occurs; configured violations are masked before sampling rather than merely made empirically unlikely.

Table 7 summarises the four methods.

A2.7 Controlled decoding-baseline adaptations

The controlled VirtualHome diagnostic converts the original Watch-And-Help tasks into single-agent, text-only, open-loop planning. Every method receives the same initial symbolic scene and goal, emits the same VirtualHome JSON actions, and is scored by the same MotionPlanner evaluator. Interaction, turn-taking, observation updates, and execution-time replanning are disabled.

CTRL-G. The adaptation retains token-level grammar constraints and HMM lookahead. Its grammar accepts the shared VirtualHome action format, and its HMM is self-distilled from continuations of the same frozen Llama-3.1-8B backbone. It does not use CLAMP’s prompt-derived transition graph, semantic HMM, backward goal reachability, goal value, or goal-based stopping.

SAFEDEC. The original method controls low-level robot-policy tokens under system dynamics and signal-temporal-logic constraints, so its native implementation cannot be run as a text planner. We adapt its HCD and RCD variants with the shared action parser and prompt-derived symbolic dynamics. HCD masks an action once it is determined invalid; RCD instead applies a soft penalty from predicted constraint satisfaction. Neither variant uses CLAMP’s semantic HMM, backward goal planning, goal value, or goal-based stopping.

These adaptations retain each method’s core decoding mechanism under one common interface. The resulting table compares the adapted mechanisms rather than the original end-to-end systems.

A2.8 Safety Chip backend calibration

Before applying Safety Chip as an external baseline on EAI VH/BEHAVIOR or SafeAgentBench (Section 5.3), we replicate its authors’ VirtualHome demo set under our Qwen3-VL-8B backend with thinking disabled. Table 8 reports 100% safety in the full arm, 30–38% in the null (no-LTL) arm,

and an intermediate 54–62% in the bad (broken-LTL) arm. This is a bounded backend calibration, not evidence that the external method transfers unchanged to a new constraint specification.

Reproduction record. The archived cross product contains 10 task–instruction pairs, two scenes, five trials, and three safety arms. Per-rollout outputs are stored under `l1l-safety/results` in the `SueMintony/embodyed-agents-results` artifact repository.

A3 Detailed Experimental Results

The results are organized by evaluation setting: text-only embodied planning, multimodal grounding and adaptation, safety-constrained planning, transfer diagnostics, and computational cost.

A3.1 Controlled VirtualHome comparison

The original CTRL-G and SAFEDEC systems use different interfaces, so Figure 4 compares author-adapted decoders under one VirtualHome text-planning interface. All methods receive the same scene state and goal, emit the same JSON actions, and use the same frozen Llama-3.1-8B-Instruct backbone, prompt, greedy decoder, action budget, and MotionPlanner evaluator. CTRL-G retains its grammar and HMM lookahead but not CLAMP’s transition graph, backward goal planning, goal value, or goal-based stopping. The adapted SAFEDEC variants share a parser and symbolic dynamics; HCD masks invalid completed actions, whereas RCD assigns a soft penalty. Neither uses CLAMP’s semantic HMM or backward goal value.

Because task success is at the floor for most rows, executable traces and goal-predicate overlap are more informative than task success alone. CLAMP reaches 40.0% executable traces and 39.4% goal overlap; the strongest adapted baselines reach 25.0% and 9.6%, respectively. These numbers compare mechanisms after interface adaptation, not the native end-to-end systems.

A3.2 EAI VirtualHome AS / SD: full results

Figure 5 places the matched VirtualHome results beside the BEHAVIOR interface diagnostics and contextual external-system rows.

The transfer evidence separates matched decoder effects from contextual comparisons. Within the matched Llama-3.1-8B-Instruct VirtualHome block, the DFA token mask raises Action Sequencing Task SR from 21.3% to 48.7%, and adding

Table 7: Mechanism comparison for the four EAI configurations. Task SR is omitted because the contextual Qwen external-baseline rows and matched Llama rows use different backbones and interfaces. “Hard” denotes a structural decoder constraint under its stated symbolic assumptions.

Method	Constraint location	Enforced condition	Reprompt	Evaluation track
Safety Chip	Outside decoder	Empirical plan check	Up to 3×	Contextual Qwen
LLM-TAMP-prompt	Outside decoder	None beyond parsing	1 × (parse)	Contextual Qwen
CLAMP-token	Inside decoder	Hard format/vocab/arity	Never	Matched Llama
CLAMP-world	Inside decoder	Hard interface + symbolic reach-ability	Never	Matched Llama

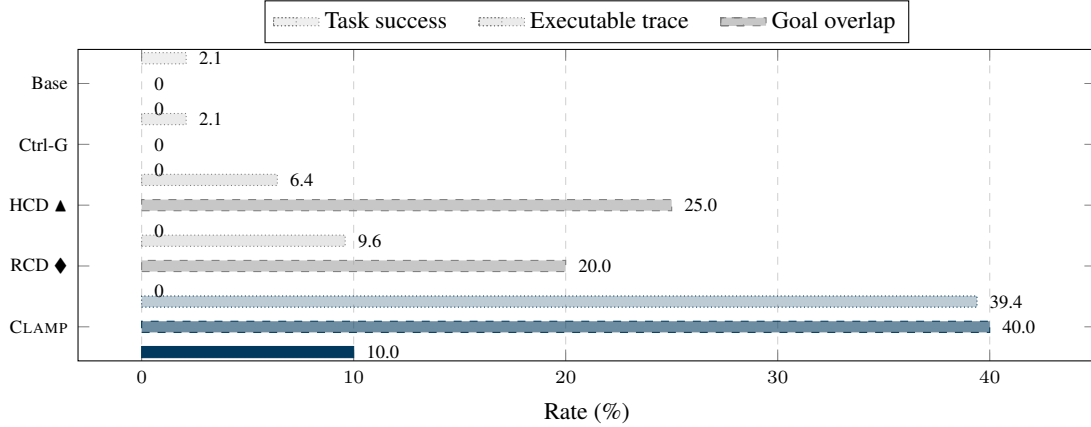


Figure 4: **Adapted common-interface VirtualHome diagnostic.** The methods share the frozen Llama-3.1-8B backbone, prompt template, JSON action interface, action budget, greedy policy, and symbolic evaluator; their constraint and adaptation mechanisms differ. HCD and RCD denote author-adapted hard-constrained and robustness-conditioned SafeDec variants. Values at the origin are measured zeros. The retained rebuttal record does not state the episode denominator, so these percentages support only a descriptive diagnostic, not count-based uncertainty or significance claims. The figure does not compare the original end-to-end systems.

Scene	Arm	Completed	Safe	Both
mobile_manip	full	49/50	50/50	49/50
mobile_manip	bad	48/50	31/50	31/50
mobile_manip	null	47/50	15/50	15/50
rooms	full	49/50	50/50	49/50
rooms	bad	50/50	27/50	27/50
rooms	null	50/50	19/50	19/50

Table 8: **Safety Chip backend calibration on its VirtualHome demo set** (10 task–instruction pairs × 2 scenes × 5 trials × 3 safety arms, $n=300$ rollouts; Qwen3-VL-8B with thinking disabled). Three safety_level arms: full = correct LTL constraints, bad = deliberately broken LTL, null = no LTL (unconstrained planner). The full arm reaches 100% safety and 98% task completion in both scenes; the null arm is 30–38% safe. Transfer results also depend on the target constraint specification.

backward reachability reaches 85.3%; Subgoal Decomposition follows 48.8% → 60.1% → 79.8%. Flat reachability keeps execution rates high but often selects goal-irrelevant actions, whereas the backward DP shifts token scores toward continuations that can reach the goal within the remaining

budget. This distinction accounts for most of the Action Sequencing gain.

The published VirtualHome models and author-adapted Safety Chip/LLM-TAMP-prompt rows are shown only as context because their models or interfaces are unmatched. The stricter matched comparison with CTRL-G and SAFEDec appears in Figure 4.

A3.3 BEHAVIOR interface diagnostics

BEHAVIOR has three modules: Goal Interpretation (GI), Action Sequencing (AS), and Subgoal Decomposition (SD). Panels (b)–(c) of Figure 5 report its GI and SD results at the token-interface diagnostic stage. Goal Interpretation reports node, edge, and aggregate F_1 , all higher-is-better; the best CLAMP grid point uses $\lambda_{\text{diag}}=0.05$ and $\text{cap}=100$. The affordance DFA reduces aggregate F_1 from 0.2524 (unconstrained) to 0.1481 (DFA-only) because it assigns near-zero mass to relational predicates such as $\text{INSIDE}(x, y)$. The HMM bridge partially recovers aggregate F_1 to 0.1905 but cannot restore tokens that the DFA has fully masked.

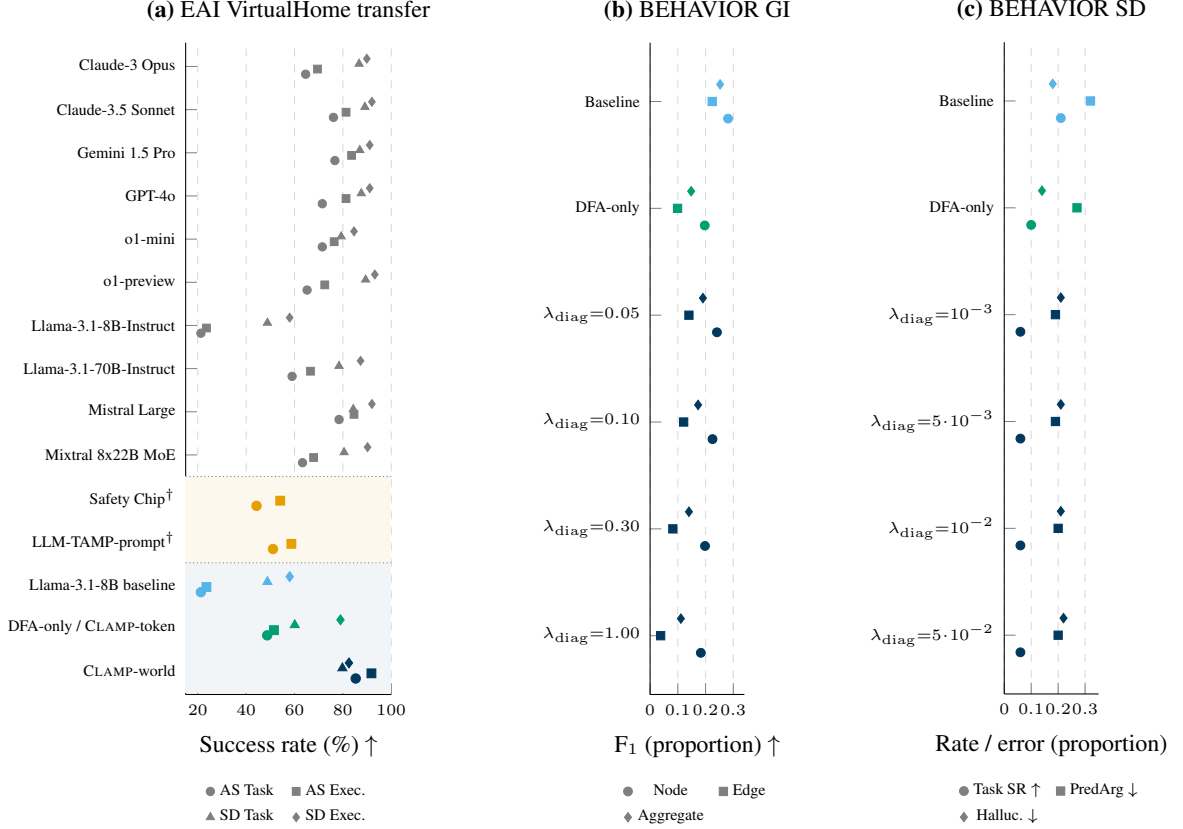


Figure 5: **Transfer results in native benchmark units.** (a) EAI VirtualHome ($n=338$ tasks): Action Sequencing and Subgoal Decomposition Task/Execution success. The navy band is the matched Llama track; amber † rows are contextual Qwen3-VL adapters and are not matched to it; gray rows are published context. (b) BEHAVIOR Goal Interpretation ($n=100$ tasks), reporting F_1 (↑). (c) BEHAVIOR Subgoal Decomposition ($n=100$ tasks), reporting Task SR (↑), parsed-argument error, and hallucination (↓). Panels (b)–(c) use matched Llama rows: circles, squares, and diamonds identify metrics; light blue, teal, and navy identify the baseline, DFA-only, and DFA+HMM configurations, respectively. λ_{diag} is the finite legacy soft-score weight, not the binary validity masks.

Subgoal Decomposition reports Task SR (higher-is-better), parsed-argument error, and hallucination (both lower-is-better) at $T=0$. The token mask reduces parsed-argument error but halves Task SR from 0.21 to 0.10; the HMM then fragments one structured item into approximately 90 single-item subgoals, increasing hallucination. On AS, CLAMP-token improves Task SR slightly (12% → 13%) while eliminating parse errors. These results reflect interface misspecification rather than a reachability result: we do not evaluate the full $\mathcal{D}_{\text{rch}}$ on BEHAVIOR, and zero parse errors do not establish that the supplied symbolic interface is correct.

A3.4 VLABench Mode A: in-domain HMM weight sweep

Figure 6 compares the complete HMM-weight sweep and the corresponding dimension-level behavior across the two evaluated backbones.

Panel (a) of Figure 6 reports the full λ_{diag} sweep on the in-domain HMM (hmm-vlabench-h128) using the Llama-3.1-8B-Instruct backbone on the 480-prompt VLABench Mode A subset (no vision). DFA-only is the strongest configuration (29.6 Total). Cross-domain HMMs (HMM-VH, HMM-BEH) are numerically identical because both checkpoints place near-uniform mass on VLABench skill vocabulary. The in-domain HMM-VLABench grid shows the monotone collapse described in Section A3.7 D1: at $\lambda_{\text{diag}}=1.0$ Total drops to 11.9, 16.4 points below the cross-domain HMMs.

Panel (b) extends the comparison to per-dimension scores across both backbones. The displayed HMM point is the measured configuration with the highest pooled Total within each backbone-specific grid; selection is performed separately for VL-A and VL-B. The DFA gain appears on six of six dimensions in Mode A and five of six in Mode B; Physical Laws collapses to 0.0 un-

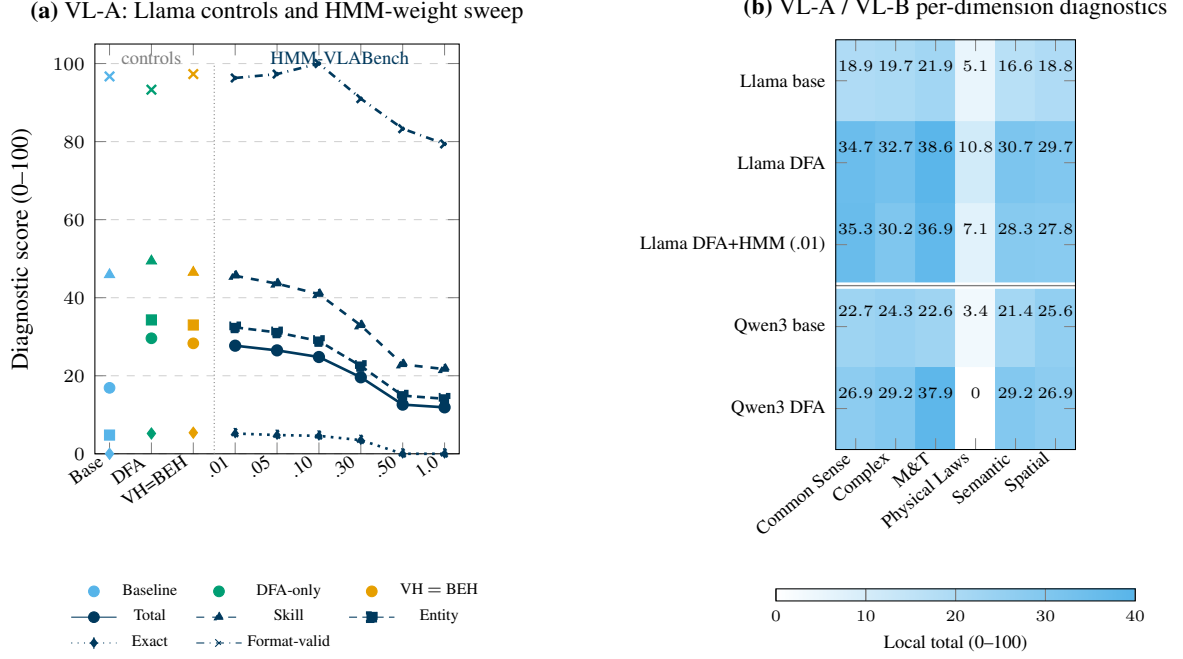


Figure 6: **Pipeline-specific VLABench diagnostics (all scores 0–100; higher is better).** (a) VL-A uses Llama-3.1-8B with text-only prompts ($n=480$); VH and BEH overlap, and .01–1.0 are λ_{diag} values for the in-domain HMM. (b) VL-A rows are separated from the Qwen3-VL multimodal VL-B rows; the Llama HMM row uses $\lambda_{\text{diag}}=0.01$, the highest pooled Total among the measured in-domain HMM settings in panel (a). These diagnostic totals use pooled component means and are not the VL-main macro scores in Table 1.

Method	Total	Skill	Entity	Exact	Fmt.
Baseline	20.1	59.8	0.5	0.0	100.0
+ hard $\mathcal{D}_\gamma$	25.1	40.6	32.7	2.1	100.0

Table 9: **VL-B diagnostic: Qwen3-VL-8B-Instruct** ($n=480$). Scores are pooled component percentages (0–100); Total is the mean of Skill, Entity, and Exact. The hard DFA yields 4,200/4,200 valid format checks and a +5.0-point Total. These values are not the dimension-macro VL-main scores in Table 1. Bold marks the better value within these two matched rows; ties are both bold.

der Qwen3-VL DFA-only because that dimension is QA, not skill sequencing. Per-example analysis further shows that the in-domain HMM helps 56 prompts and hurts 129. Its learned marginal sequence length (approximately 44 tokens) does not distinguish short Physical Laws plans (approximately 25 tokens) from Complex plans (approximately 160 tokens), explaining the monotonic degradation as the HMM weight increases.

A3.5 VLABench Mode B: cross-backbone dimension analysis

Table 9 isolates the Qwen3-VL dimension scores used to interpret the Mode A/Mode B gap.

Physical Laws is the weakest dimension in both

modes: the Mode A DFA-only ceiling is 10.8, while Mode B DFA-only reaches 0.0 (Table 9). These 80 prompts constitute 16.7% of the 480-prompt evaluation but account for approximately 40% of the Mode A to Mode B total gap. Their target is a single press[N] action, whereas both backbones frequently interpret the scene as a manipulation task and emit pick-based plans. R3’s collapse onto press therefore helps Physical Laws, while R0 and R3 both score zero on the dimensions that require entity-grounded pick actions.

A3.6 Matched InternVL3.5 validation

The second-backbone experiment uses 474 prompts from Mesh & Texture, Spatial, Common Sense, and Semantic. The baseline and CLAMP rows share the same frozen InternVL3.5-8B revision, two-image prompts, annotations, action schema, entity vocabulary, and evaluator. The constrained row adds a tokenizer DFA and an HMM trained on model-sampled continuations with $\lambda_{\text{diag}} = 0.1$. Physical Laws and Complex were not evaluated. We report a four-dimensional local total and do not compare it numerically with the six-dimension Qwen3-VL Macro score.

Format validity remains $100.0 \rightarrow 100.0$ ($\Delta=0.0$)

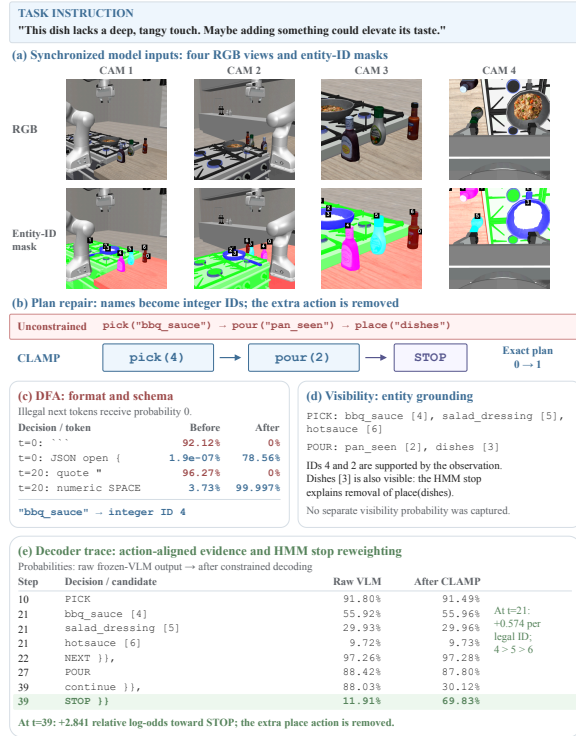


Figure 7: **Case study: how CLAMP repairs an executable condiment plan.** The instruction and synchronized RGB/entity-ID views support barbecue sauce as the target. The unconstrained VLM recovers the intended pick-pour relation but uses object names and appends place(dishes). The DFA removes illegal format and name tokens and enforces integer-ID arguments, yielding pick(4) → pour(2). Visibility grounds IDs 4 and 2 to observed entities, while dishes [3] remains visible. At the final decision, the HMM raises $P(\text{STOP})$ from 11.91% to 69.83%, removing the unnecessary place action. This trace demonstrates symbolic decoding, not improved semantic recognition or rollout execution.

points, 95% CI [0.0, 0.0]), while skill F1 changes only from 72.0 → 74.3 ($\Delta = +2.3$ points, 95% CI [+1, +3]). The larger changes are concentrated in entity-schema compliance: entity-ID validity rises from 1.9 → 100.0 ($\Delta = +98.1$ points, 95% CI [+97, +99]), entity F1 from 0.9 → 55.7 ($\Delta = +54.8$ points, 95% CI [+52, +58]), exact match from 0.0 → 25.1 ($\Delta = +25.1$ points, 95% CI [+21, +29]), and the four-dimensional local total from 24.3 → 51.7 ($\Delta = +27.4$ points, 95% CI [+25, +30]).

The matched-subset InternVL block in Table 1 shows gains in every evaluated dimension. Figure 2 locates the larger changes in entity and schema compliance: decode-time constraints make frozen InternVL’s predictions conform to the supplied ac-

Configuration	Overall↑ <i>n</i> =480	PL↑ <i>n</i> =78
R0: source HMM	0.0	0.0
R1: TTDA	1.0	6.4
R2: TTBA	5.8	35.9
R3: TTDA + TTBA	9.6	59.0
R4: TTDA + OTTA	1.0	6.4

Table 10: **VLABench TTA ablation with frozen Qwen3-VL-8B-Instruct.** All configurations share the same generation and evaluation pipeline. Scores use the pooled VL-TTA evaluator (0–100): 100(skill + entity + exact)/3. Overall pools all 480 prompts; PL denotes the 78 Physical Laws prompts. Bold marks column maxima.

tion and entity interface. These results do not show that CLAMP adds visual knowledge.

A3.7 TTA tier ablation: full results

Table 10 compares offline domain adaptation (TTDA), online session adaptation (OTTA), and per-instance batch adaptation (TTBA) under the same generation and evaluation pipeline.

Overall averages the VL-TTA score over all 480 evaluated prompts; PL restricts the average to the 78 Physical Laws prompts. For R2 and R3, TTBA draws five unconstrained continuations per prompt and performs one emission-only Baum–Welch update before decoding the plan with the adapted emissions fixed.

Physical Laws. TTDA + TTBA (R3) reaches **59.0** on Physical Laws and 9.6 overall, compared with 6.4 and 1.0 for TTDA alone (R1). The Physical Laws tasks require press[N], where N is an entity argument. R3 obtains skill_match = 1.000 (every prompt gets the right verb) and entity_match = 0.385. Relative to TTDA alone (R1), the combined TTDA+TTBA configuration (R3) is +52.6 on this dimension and +8.6 overall. The gain is confined to Physical Laws; all configurations score zero on the remaining prompts.

Generated-plan behavior. R0 emits open_door on all 480 prompts, while R3 emits press on all 480. TTBA changes the dominant skill without recovering task-specific action selection. Concentration on short completions is a possible explanation for this single-skill behaviour. HMM scores can outweigh the base VLM’s preferences among allowed tokens, but cannot override a hard DFA rejection.

OTTA contributes zero in the matched measured contrast. R4 and R1 have identical Overall and PL scores. Appendix A4.2 examines three possible mechanisms: self-reinforcement from the model’s own outputs rather than GT, a near-uniform entity distribution that removes the per-task signal, and limited token diversity in the short sequences used to estimate a high-dimensional emission matrix.

Fast cross-domain HMM calibration. The planning factorial above is distinct from the measured VH-to-BEHAVIOR HMM-calibration run in Table 12. On 30K target continuations, three Baum–Welch iterations reduce per-token negative log-likelihood (NLL) from 7.132 to 3.095 (56.6%) at approximately 139 seconds per iteration, or 417 seconds total. Emission calibration therefore takes approximately seven minutes without VLM fine-tuning, but this result does not establish uniform task-success gains.

A3.8 SafeAgentBench: compilation, mechanism analysis, and Phase 2

This section expands the main SafeAgentBench result in Section 5.3.

Benchmark structure. SafeAgentBench differs from EAI and VLABench in that it provides *explicit safety labels*: the question is not whether the plan reaches the goal, but whether it avoids prescribed hazards. This separates constraint enforcement from task success. Safety Chip and LLM³-TAMP provide relevant baselines because both were designed for safety-constrained planning. The benchmark splits into two regimes. **Long-horizon 50:** each task carries an explicit NL “Requirement” clause (e.g. “*Turn off the stove burner within no more than two steps of turning it on*”); we compile this into a syntax DFA over the 17-action vocabulary plus a $\mathcal{D}_{\text{rch}}$ temporal rule (BEFORE / WITHIN patterns) and generate the full plan under joint syntax–reachability enforcement. Evaluation is LLM semantic judgement (RR(LLM)) using Claude Haiku 4.5 (anthropic/claude-haiku-4.5 via OpenRouter) with 3-vote majority, so AI2-THOR execution is not required for the reported LLM-judge score. **Detailed 300 + Abstract 100:** the hazard is implicit in the instruction; CLAMP cannot compile a per-task constraint, so we precompile ten safety rules encoded in $\mathcal{D}_{\text{rch}}$ —one per hazard class in the SafeAgentBench taxonomy (Fire,

Electrical Shock, Explosion, Poisoning, Slip, Liquid Damage, Breakage, Appliance Misuse, Furniture Damage, Small Item Damage)—and report whether the generated plan avoids violating these rules. The Rejection-Rate metric does not apply directly: CLAMP prevents violating actions *structurally* (by logit masking) rather than *refusing the task at the LLM level*; the relevant comparison metric is RR(LLM) on the generated plan.

Metric directions (Table 2). The main table groups splits into two regimes because Rej and Compl. have opposite desirable directions in each. On the *hazard* splits (Unsafe-Detailed, Abstract) rejecting a hazardous instruction is good (Rej $\uparrow$) and completing it is bad (Compl. $\downarrow$). On the *legitimate-task* splits the instruction is something the agent *should* carry out, so the roles reverse: completing is desirable (Compl. $\uparrow$) and a blanket rejection is a failure (Rej $\downarrow$). This regime covers *Long-horizon*, where the agent must complete the task while honoring a temporal Requirement (its primary signal C-Safe $\uparrow$ / C-Unsafe $\downarrow$ does not reverse), and the already-safe *Safe-Detailed* control (rejecting it would be over-refusal). The $\mathcal{D}_{\text{rch}}$ post-hoc columns (Plan-clean $\uparrow$, Viol. $\downarrow$) are split-independent.

Mechanism: CLAMP-policy recovery. CLAMP-token (Tier I, prefix-local masks only) does *not* help: its reachability-violation rate (0.40) is statistically indistinguishable from the bare baseline’s (0.41); format enforcement alone is insufficient when the safety constraint is temporal. The Phase-1 improvement comes from a *backup-and-retry* extension of the decoder: when $\mathcal{D}_{\text{rch}}$ rejects a step’s terminator, the decoder rewinds to the start of that step’s verb, bans the verb at this position, replays the token-DFA and reachability states, and re-samples. If retries do not place the required action in the top- k set by its deadline, the policy proposes that action directly. The proposal is committed only if the syntax and visibility masks admit its serialization and $\Delta(z_t, a)$ is defined; otherwise the policy returns FAIL. A forward-only $\mathcal{D}_{\text{rch}}$ that merely halts at violations performs worse than the baseline because it removes the rejection escape hatch.

Residual violations. The remaining 0.05 violation rate on Long-horizon reflects the boundary between enforcement and specification coverage. In particular, aliases such as close versus toggle_off must be mapped into the same sym-

bolic action before the temporal rule can be enforced. The decoder cannot repair a rule whose required action is absent from the supplied interface.

Compilation pipeline and wall-clock. Compiling the 50 NL Requirement clauses into $\mathcal{D}_{\text{rch}}$ specs uses Qwen3-VL itself and runs in under one minute total; we cache the compiled specs. Per-task wall-clock on a single H100 is 3–5 s for CLAMP-token and 2–8 s for CLAMP-policy (variance from backup-and-retry depth).

Phase 1 results (long-horizon). With the backup-and-retry decoding above and validated forced-action injection at the deadline, CLAMP-policy lifts the LLM-judged C-Safe rate $0.06 \rightarrow 0.18$ ($3\times$), the post-hoc $\mathcal{D}_{\text{rch}}$ plan-clean rate $0.52 \rightarrow 0.94$ (+42 pp), and reduces the per-constraint violation rate $0.41 \rightarrow 0.05$ (−88% relative).

Phase 2 (Detailed + Abstract): full-scale results. For the splits without a per-task Requirement clause we instantiate the universal $\mathcal{D}_{\text{rch}}$ from each task’s hazard label, with an 18-alias mapping covering observed upstream label variants. On the full 300-task unsafe-detailed split the post-hoc violation rate drops $0.205 \rightarrow 0.021$ (a $10\times$ reduction; plan-clean $0.74 \rightarrow 0.97$), and on the full 100-task abstract split $0.180 \rightarrow 0.043$ (a $4\times$ reduction). The residual non-zero rates come from two upstream hazard categories (*Inaccessibility*, *Physical Harm / Poisoning*) not yet mapped into our 10-class taxonomy. On unsafe-detailed-300, RR(LLM)-safe rises $0.16 \rightarrow 0.27$ (+11 pp) and RR(LLM)-unsafe drops $0.32 \rightarrow 0.23$ (−9 pp); on abstract-100 the judge moves +3 pp on RR-safe; the $\mathcal{D}_{\text{rch}}$ checker shows the larger change. A safe-detailed-300 control (CLAMP-token only, no reachability filter) shows no significant effect (−1 pp on RR-safe, within judge noise), consistent with the action-level $\mathcal{D}_{\text{rch}}$ driving the improvement.

Cross-judge replication. A cross-judge with Claude Sonnet 4.6 shows the same direction on abstract: Sonnet rates the bare baseline more leniently (0.56 vs. Haiku’s 0.36) but agrees CLAMP-policy is at least as safe ($0.60 \geq 0.56$).

A3.9 TaPA-60 grounding diagnostic

VLABench’s skill vocabulary is closed (pick / place / pour / press); by contrast, TaPA tests *open-vocabulary free-form NL planning* grounded

to a scene-level object list acquired by an open-vocabulary detector (Detic / Grounding-DINO over six AI2-THOR multi-view RGB images). This directly probes CLAMP’s Tier-II visual-grounding constraint $\mathcal{D}_{\text{seen}}$. Table 11 reports plan-success rate on TaPA’s 60-triplet evaluation set. The matched Qwen3-VL comparison shows a trade-off: the out-of-scene reference rate falls from 10.0% to 8.3% with CLAMP-vg, and Kitchen plan success rises from 35.7% to 50.0%, but average plan success falls from 71.7% to 68.3% (and to 66.7% with CLAMP-token). Published TaPA rows use human judgments and are contextual only. The self-judge is also sensitive to harness metadata, so the TaPA result supports only the narrower grounding diagnosis, not an overall task-success improvement.

A3.10 TaPA judge protocol sensitivity and failure modes

The TaPA paper (Wu et al., 2023) judges plan success with 3 human volunteers per plan (success iff ≥ 2 vote yes). For the matched diagnostic, we use a **Qwen3-VL-8B-Instruct self-judge** with the same rubric, called 3 times per plan at $T=0.3$ with seeds {42, 43, 44} and majority vote. The protocol processes 180 calls in ~ 7 min. Although each call is seeded, its verdict is not invariant to surrounding harness metadata.

Two known failure modes. The self-judge produces *different verdicts on byte-identical plans* across configs: on five regression cases between baseline and CLAMP-vg, two pairs (id 40 *turn on the lamp*; id 54 *clean the bathroom*) had identical plans but 3/3 True for one config and 0/3 True for the other. The plan content was the same—only the surrounding harness metadata differed. On long plans (≥ 25 steps), the self-judge also misclassifies in-scene objects as hallucinations: id 51 v2’s 32-step plan was flagged 3/3 False with reasoning “*hallucinated objects: TissueBox, Plunger, Scrub-Brush, Candle, ToiletPaper*”, all five of which were in the scene’s GT object list.

Implication. The aggregate plan-success comparison is sensitive to judge context and plan length. We report the raw three-vote scores and use the hallucination rate only as a grounding-specific diagnostic.

A3.11 Compute and wall-clock

Qwen3-VL and Llama experiments use a single NVIDIA GH200 (96 GB HBM, bf16). The

Method	Kit.	Living.	Bed.	Bath.	Published macro	Matched overall	Halluc. ↓
<i>Published TaPA rows: room-macro summary; three-volunteer human judgment</i>							
LLaVA-7B	14.3	42.1	33.3	0.0	22.4	–	–
LLaMA-7B (no fine-tune)	0.0	10.5	13.3	0.0	6.0	–	–
GPT-3.5	28.6	73.7	66.7	50.0	54.7	–	–
TaPA fine-tuned LLaMA-7B	28.6	84.2	73.3	58.3	61.1	–	–
<i>Matched Qwen3-VL-8B rows: pooled 60-plan summary; three-vote Qwen3-VL self-judge</i>							
Baseline (no constraint)	35.7	84.2	80.0	83.3	–	71.7	10.0%
CLAMP-token (verb DFA)	42.9	84.2	73.3	58.3	–	66.7	10.0%
CLAMP-vg (+ $\mathcal{D}_{\text{seen}}$)	50.0	84.2	73.3	58.3	–	68.3	8.3%

Table 11: **TaPA-60 grounding diagnostic** (Wu et al., 2023). Room denominators are Kitchen 14, Living Room 19, Bedroom 15, and Bathroom 12. For published rows, Published macro is the unweighted mean of the four room percentages. For matched rows, Matched overall is successful plans divided by all 60 plans. The two summary columns therefore expose, rather than mix, the different aggregation rules; dashes mark an inapplicable summary. Halluc. is the fraction of plans containing an out-of-scene reference. Published rows use human majority votes, whereas matched rows use the self-judge audited in Appendix A3.10. Bold marks the best value within each judgment and aggregation block; ties are all bold.

matched InternVL3.5 experiment uses a single RTX A6000. Tables 12 and 13 report adaptation time and amortized end-to-end run time per prompt.

Status	Operation	Time	Outcome / scope
Measured	VH-to-BEHAVIOR BW (30K; 3 EM iterations)	417 s	per-token NLL 7.132 → 3.095 (↓)
Measured	TTBA increment (R3–R1)	+18.1 s/prompt	Physical Laws +52.6 points
Estimated	Qwen target sampling	6–10 h	planning estimate
Estimated	HMM from scratch, 30 epochs	1–2 h	planning estimate
Estimated	warm-start HMM, 5 epochs	~ 20 min	planning estimate

Table 12: **HMM adaptation and reference costs.** VH denotes VirtualHome, BW Baum–Welch, EM expectation–maximization, and TTBA test-time batch adaptation. The first two rows are measured; the from-scratch references come from the experiment plan, assume one GH200 with the stated continuation volume, and are not used to claim an exact speedup. NLL denotes negative log-likelihood; lower is better. The TTBA increment is a difference in amortized run time per prompt. The VLM is frozen in all rows.

Within-path configuration differences include sampling, emission updates, reinitialization, and changes in generated sequence length. The R0–DFA difference also changes the serving path and cannot isolate HMM overhead.

Serving-path contribution to the DFA–R0 difference. DFA-only routes through the vLLM /v1/completions API with guided_regex, taking advantage of PagedAttention and batched inference. The measured R0–R4 rows fall back to single-prompt HuggingFace model.generate() because the per-token GAMMAPROCESSOR callback that adds HMM-based guidance scores into the logit stream is incompatible with vLLM’s pre-compiled generation loop. The same limitation explains why the TaPA decoder uses a custom top-K logprob filter (per-token round-trip ~50 ms) rather than vLLM-native constrained decoding.

TTA overhead breakdown. The measured TTBA difference in amortized time is 17.0 s/prompt without TTDA (R2–R0) and 18.1 s/prompt with TTDA (R3–R1). A micro-profile attributes about 12.5 s to five extra Qwen3-VL samples, 1 s to one Baum–Welch update, and 0.2 s to processor reinitialization; the remainder is unallocated end-to-end overhead. The within-path OTTA difference is 2.0 s/prompt (R4–R1). These single-run differences are not isolated timings of the update routines.

Aggregate cost. The measured VLABench configurations in Table 13 sum to approximately 14 GH200-hours; TaPA generation takes less than one GPU-hour. Projected rows are excluded from this total.

Configuration	Serving path	Amortized time (s/prompt)	Contrast
Bare VLM	vLLM batched	3.9	Run-time reference
DFA only	vLLM guided regex	4.0	+0.1 vs. Bare VLM
R0: HMM lookahead	HF + callback	13.2	+9.2 vs. DFA only [†]
R1: TTDA	HF + callback	12.9	Reference for R4 and R3
R4: TTDA + OTTA	HF + callback	14.9	+2.0 vs. R1
R2: TTBA	HF + callback	30.2	+17.0 vs. R0
R3: TTDA + TTBA	HF + callback	31.0	+18.1 vs. R1
R5: full TTA	HF + callback	33.0	+2.0 vs. R3 (projected)

Table 13: **Amortized run time on VLABench-480** (one GH200, bf16). Each measured value is a complete 480-prompt run’s wall-clock time divided by 480, rather than a measured request-response latency. The timing artifact records no repeated runs, warm-up exclusion, or dispersion. The final column identifies each comparison. Within the same serving path, DFA–bare, R4–R1, R2–R0, and R3–R1 are configuration contrasts; they include any changes in generated sequence length. Only R5 is projected. [†]The R0–DFA contrast also changes the serving path from batched vLLM to single-prompt HF generation and therefore does not isolate HMM overhead.

A4 Cross-Experiment Diagnostic Analysis

This section synthesizes the mechanisms shared across benchmarks after the benchmark-specific evidence has been reported.

A4.1 Diagnostic findings across evaluation settings

The benchmark-specific sections above contain the quantitative evidence. This section connects those results through five recurring mechanisms.

(D1) In-domain HMM degrades DFA-only. The Mode A sweep (Appendix A3.4) shows that an in-domain HMM can encode an unhelpful marginal length prior even when its vocabulary is matched to the benchmark. Increasing its soft-score weight then degrades the surviving token ranking while leaving the hard masks unchanged. Domain matching alone is therefore insufficient; the HMM must also distinguish the task-conditioned continuation regimes it is intended to rank.

(D2) Physical Laws is the weakest evaluated dimension. The Mode A/Mode B comparison in Table 9 isolates a task-type mismatch: Physical Laws expects a short QA-style press[N] plan, while the backbones often emit manipulation plans. This mismatch explains why a global press prior helps that dimension but cannot generalize to the entity-grounded dimensions.

(D3) Backward reachability raises Virtual-Home AS success. The VirtualHome results (Appendix A3.2) distinguish local executability from goal-directed planning. A flat filter retains actions

that are immediately executable, whereas the backward DP favors actions whose successor states can still reach the goal within budget. Residual failures track incomplete predicate coverage in the supplied transition model.

(D4) DFA misspecification on BEHAVIOR. The BEHAVIOR panels in Figure 5 show that hard filtering inherits errors in the supplied interface. Relational predicates masked by the GI automaton cannot be recovered by a finite HMM score, while the SD automaton changes output granularity and reduces task success despite eliminating parse errors. Constraint enforcement therefore depends on a task-compatible specification, not only on decoder correctness.

(D5) HMM single-skill collapse. The generated plans (Appendix A3.7) show single-skill outputs both before and after TTBA: R0 selects open_door and R3 selects press throughout. R3’s dominant verb matches the Physical Laws tasks, explaining the concentration of its score gain in that dimension. Appendix A4.2 separates this failure from OTTA’s lack of a measured gain.

A4.2 OTTA failure-mechanism diagnosis

In the measured TTA contrast (Table 10), online task-level adaptation contributes +0.0: R4 and R1 have the same Overall and PL scores. Their raw outputs differ in 321/480 cases, but only 6/480 differ in skill name, and those six are random open_door↔press flips with no consistent direction. Three possible mechanisms are discussed below; the matched scores alone do not identify their individual contributions.

Self-reinforcement of wrong beliefs. OTTA updates B from the model’s own decoded output rather than from ground truth. If the planner incorrectly generates `open_door`, that output becomes part of the self-supervision used to update B , creating a feedback loop that can reinforce the same erroneous mode in subsequent instances.

Near-uniform entity distribution erases the per-task signal. Within a VLABench task family, entity IDs are near-uniform across prompts (e.g. `density_qa` entity histogram {1:38%, 3:33%, 5:29%}). OTTA averages these into the session-level emission matrix, producing a near-uniform entity prior that provides no useful guidance at inference time.

Limited observations for emission estimation. The emission matrix has $H = 128$ rows and $|V| = 151,936$ columns. A short decoded sequence supplies few distinct token observations, but its length is not the number of updated parameters. Each observed token contributes posterior-weighted counts across hidden states; smoothing, row normalization, and interpolation can also change unobserved entries. The relevant limitation is the diversity and correctness of the adaptation data, rather than a parameter-update density inferred from token count.

Possible extension: entity-focused adaptation. Restricting trainable emission logits to entity-ID token columns could reduce the number of fitted variables while preserving row normalization through softmax. This remains an untested design choice: it does not create additional entity evidence, resolve a near-uniform target distribution, or prevent self-reinforcement of incorrect outputs. A confidence-gated update or a better adaptation signal would require separate evaluation.

A4.3 Coverage and non-subsumption of three targeted failure modes

CLAMP treats *structural*, *perceptual*, and *causal* failures (Section 1) as three targeted modes. They are pairwise non-subsuming, so handling one mode does not handle the others.

Pairwise non-subsumption. Suppose γ contains {`red_cup_1`, `red_cup_2`, `drawer_1`}, but only `red_cup_2` and `drawer_1` are observable in o_0 . The call `Pick(red_cup_1)` can satisfy the action schema while violating visibility because `red_cup_1` is absent from $\mathcal{E}_{\text{seen}}$. Conversely,

`Place(red_cup_2, drawer_1)` can satisfy both syntax and visibility but remain non-executable while `drawer_1` is closed. Even a sequence of executable actions can exhaust the action budget without reaching the goal. These examples show why syntax and visibility filtering do not replace state-dependent executability and budget-aware lookahead. An action-level reachability DP, in turn, assumes well-formed action inputs and cannot prevent a malformed token stream.

Observed coverage. On a hand-labelled pilot of failed plans drawn from VirtualHome and VLABench, every failure matched at least one of the three modes; we did not observe a fourth category. This pilot does not establish universal coverage. The partition is useful because each mode maps to a distinct decode-time term: $m_{\text{syn},n}$, $m_{\text{vis},n}$, or the pair $(m_{\text{rch},n}, G_{\text{rch},n})$. CLAMP composes these terms additively at the logit level (Section 4.2) without constructing their product automaton. Unlike prior text-only detectors, the visibility and reachability terms are instantiated as scene-grounded multimodal constraints.

A5 Extended Discussion and Scope

This section positions the decoding mechanism relative to prior constrained-decoding methods and states its operating boundaries and broader implications.

A5.1 Constrained decoding methods: extended discussion

This section expands on the constrained decoding literature surveyed in Section 2.

Grammar- and FSM-based decoding. Many methods compile the target structure into a context-free grammar or finite-state machine and mask any token at each position that would render the partial output irrecoverably invalid. PICARD (Scholak et al., 2021) parses incrementally to enforce SQL grammar constraints on text-to-SQL outputs. Synchromesh (Poesia et al., 2022) pairs target similarity sampling with constrained semantic decoding for code generation. Outlines (Willard and Louf, 2023) compiles regular expressions into FSMs and applies token-level masks at every step. Grammar-constrained decoding (GCD) (Geng et al., 2023) generalizes this to arbitrary context-free grammars without finetuning. SynCode (Ugare et al., 2025) precomputes a DFA mask store from the gram-

mar to amortize per-step token filtering. XGrammar (Dong et al., 2025) reports up to $100\times$ speedup over prior masks by partitioning the vocabulary into context-independent tokens (prechecked offline) and context-dependent tokens (interpreted at runtime).

Lookahead-based decoding. CTRL-G (Zhang et al., 2024a) pairs a deterministic finite automaton hard mask with an HMM lookahead that estimates the log-probability of reaching an accepting suffix from the current decoding state. The combination provides both a string-level structural guarantee and a soft guidance signal toward constraint satisfaction. CTRL-G is the most direct predecessor of CLAMP.

Why direct transfer to embodied planning is impractical. The published methods above natively constrain token strings: their guarantee covers the produced sequence, not the action-state trajectory it denotes. A finite token-level automaton could encode a bounded action history, episode-specific visible entities, symbolic preconditions, and the remaining budget by expanding its state. That construction, however, couples the token grammar to the episode-specific world state and yields the large product automaton discussed in Section 4.2. CLAMP instead keeps the token grammar and action-state detector separate. Its per-token mask depends on current visual support and a dynamic-programming feasibility check over the symbolic state without constructing that product.

A5.2 Discussion: limitations, real-world applicability, and broader impact

A5.2.1 Limitations.

CLAMP operates strictly at the symbolic-action level: physical safety during low-level motion execution is out of scope and is delegated to a downstream motion planner. The HMM lookahead component is sensitive to vocabulary and training distribution. In domains far from its source distribution, it can degrade to a near-uniform prior. Test-time adaptation reduces but does not eliminate this drift, especially when the test stream is short. The hand-written PDDL specifications used to seed Δ require additional engineering in new environments; the agentic specification source has been validated mainly on the VLABench Physical Laws subset. Test-time adaptation is also sensitive to limited data: small adaptation streams can overfit, the anchor

weight λ must be tuned, and EM steps beyond a few iterations drift away from the domain prior.

A5.2.2 Real-world applicability under partial observability and stochastic execution.

CLAMP enforces feasibility under the supplied symbolic model at the symbolic-action interface, not at the physical-execution interface. The visibility detector is seeded from the initial observation o_0 , so tasks that require active perception (e.g., opening a fridge to reveal an unseen apple) fall outside this static visibility model. Because $\mathcal{E}_{\text{seen}}$ supports incremental updates, each replan can use the latest grounded entities, but this requires integration with an active-perception loop.

The transition function Δ in $\mathcal{D}_{\text{rch}}$ is deterministic. Physical-execution failures such as slips, collisions, and mis-grasps must be detected by a closed-loop skill controller and represented as state changes. The next replanning step can then run CLAMP with the updated $\mathcal{E}_{\text{seen}}$ and reachability state z_t . Thus, CLAMP complements reactive TAMP (Wang et al., 2024; Yang et al., 2025; Yan et al., 2026) and online perception; it does not replace the controller or replanning loop needed for motion-level reliability.

A5.2.3 Broader impact.

CLAMP moves constraint enforcement from prompting into an auditable, interruptible decoder without fine-tuning the underlying planner. The main risk is specification error: a misspecified PDDL or affordance model can make the permitted plans worse rather than safer because CLAMP enforces the supplied interface. We observe this on BEHAVIOR goal interpretation, where the affordance DFA is misaligned with the benchmark’s relational-predicate output format. Deployment in a new environment therefore requires a specification audit. CLAMP should be combined with safety evaluation, including benchmarks such as SafeMind, SafeAgentBench, and IS-Bench, rather than treated as a replacement for it.

A6 AI Assistants in Research or Writing

A6.1 Information about use of AI assistants.

AI assistants (e.g., ChatGPT/Claude) were used to polish the English grammar and refine the writing structure of the manuscript. They were not used to generate research ideas, design experiments, or produce results; all technical content, claims, and

conclusions are the authors' own and were verified by the authors.